

Backend Development: Practical Engineering for Production Systems

Diogo Ribeiro

February 18, 2026

Contents

1	Introduction	1
1.1	A reference system	1
1.2	What “good” looks like	1
1.3	Scope and non-goals	1
1.4	A minimal API surface	1
1.5	Quality attributes	2
1.6	The operating model	2
1.7	A small data model	2
1.8	Change as the primary constraint	2
1.9	Common failure modes	2
1.10	Working style	3
1.11	Who this is for	3
1.12	System boundaries	3
1.13	A sample request	3
1.14	Operational baseline	3
1.15	Documentation artifacts	3
1.16	What a backend is responsible for	4
1.17	How backends fail in practice	4
1.18	A short architecture sketch	4
1.19	Development lifecycle	4
1.20	Tradeoffs are expected	4
1.21	From prototype to product	4
1.22	Operational constraints	4
1.23	Interfaces beyond HTTP	5
1.24	Release quality gates	5
1.25	Who depends on you	5
1.26	Reference workflow	5
1.27	Baseline metrics	5
1.28	Developer experience	5
1.29	Case study: growth without chaos	5
1.30	Configuration and environments	6
1.31	Feature flags	6
1.32	Data contracts	6
1.33	Incident response	6
1.34	Scaling the team	6
1.35	Operational posture	6
1.36	Production constraints you cannot ignore	6
1.37	API evolution mindset	6
1.38	Example ownership map	7
1.39	Release reliability	7

1.40	Assumptions and load profile	7
1.41	Security baseline	7
1.42	Cost awareness	7
1.43	Dependency management	7
1.44	Architecture decisions	8
1.45	Readiness review	8
1.46	Operational drills	8
1.47	Service catalog	8
1.48	Risk register	8
1.49	Definition of done	8
1.50	When to split services	8
1.51	Operating principles	8
1.52	Stakeholder alignment	9
1.53	Example release plan	9
2	The backend mental model	10
2.1	Request lifecycle	10
2.2	A concrete flow	10
2.3	State and side effects	10
2.4	Dependencies	10
2.5	Failure modes	11
2.6	Backpressure	11
2.7	Concurrency	11
2.8	A dependency budget	11
2.9	State machines	11
2.10	Idempotency	11
2.11	Idempotency key example	11
2.12	Timeouts and deadlines	12
2.13	Synchronous vs asynchronous work	12
2.14	Data consistency boundaries	12
2.15	Failure taxonomy	12
2.16	Recoverability	12
2.17	Load and saturation	12
2.18	Cache strategy	12
2.19	Command and query separation	12
2.20	Failure budget example	12
2.21	Deployment as part of the model	13
2.22	Read path vs write path	13
2.23	Invariants	13
2.24	Race conditions	13
2.25	Optimistic locking	13
2.26	Critical path example	13
2.27	Outbox pattern	13
2.28	Sagas and orchestration	14
2.29	Graceful degradation	14
2.30	Observability targets	14
2.31	Request timeline	14
2.32	Consistency ladder	14
2.33	Coordination patterns	14
2.34	Replay and recovery	14
2.35	Observability as feedback	14
2.36	Incident example	15

2.37 SLOs and error budgets	15
2.38 State and isolation	15
2.39 Queueing and asynchronous guarantees	15
2.40 Timeouts, retries, and budgets	15
2.41 Burst handling	15
2.42 Freshness and staleness	15
2.43 Multi-tenant boundaries	16
2.44 Batch versus online	16
2.45 Traffic classes	16
2.46 Backpressure signals	16
2.47 Jittered retries	16
2.48 Request cancellation	16
2.49 Single-writer principle	16
2.50 Latency budget worked example	16
2.51 Tracing as a model	17
2.52 Capacity planning	17
2.53 Dependency matrix	17
2.54 Queue sizing	17
2.55 Retry budgets per dependency	17
2.56 Case study: timeout cascade	17
2.57 Saturation checklist	18
3 HTTP that matters	19
3.1 Methods and semantics	19
3.2 Status codes	19
3.3 Status code mapping	19
3.4 Idempotency	20
3.5 Headers that matter	20
3.6 Idempotency in practice	20
3.7 Pagination	20
3.8 Caching headers	20
3.9 Content negotiation	21
3.10 Safe and unsafe methods	21
3.11 Streaming responses	21
3.12 Request size limits	21
3.13 Problem details	21
3.14 Error response example	21
3.15 Request body size	21
3.16 Client retry behavior	21
3.17 Caching strategy	21
3.18 Content type versioning	22
3.19 Conditional requests	22
3.20 ETag example	22
3.21 Rate limiting	22
3.22 Retry-After	22
3.23 Timeouts	22
3.24 Compression	22
3.25 Correlation identifiers	22
3.26 Example request	22
3.27 Example response	23
3.28 Health and readiness endpoints	23
3.29 Batch endpoints	23

3.30	CORS	23
3.31	Security headers	23
3.32	Authentication patterns	23
3.33	Authorization errors	23
3.34	Request validation	23
3.35	Partial responses	24
3.36	Webhooks and callbacks	24
3.37	Long-running work	24
3.38	Error budget for clients	24
3.39	Input size and pagination limits	24
3.40	Conditional request example	24
3.41	ETag and If-Match for writes	24
3.42	File uploads	24
3.43	HTTP caching workflow	25
3.44	Problem details with metadata	25
3.45	Header budget	25
3.46	Response shape stability	25
3.47	Cache-Control directives	25
3.48	HTTP response example	25
3.49	Cookies and sessions	25
3.50	Redirects	26
3.51	Method safety matrix	26
3.52	Content negotiation example	26
3.53	Link headers for pagination	26
3.54	Range requests	26
3.55	HTTP error response example	26
3.56	Throttling strategy	26
3.57	Batch request example	27
4	API design	28
4.1	Resources and nouns	28
4.2	Stable identifiers	28
4.3	Naming conventions	28
4.4	Nested resources	28
4.5	Versioning	28
4.6	Versioning strategies	28
4.7	Deprecation	29
4.8	Error format	29
4.9	Field naming	29
4.10	Filtering	29
4.11	Expandable fields	29
4.12	Partial responses	29
4.13	Idempotent creates	29
4.14	Pagination and filtering	29
4.15	Pagination example	29
4.16	Filtering example	30
4.17	Sorting	30
4.18	Partial updates	30
4.19	Long-running operations	30
4.20	Job example	30
4.21	Resource lifecycle	30
4.22	Documentation	30

4.23	Compatibility testing	31
4.24	Endpoint naming rules	31
4.25	Change policy	31
4.26	Search endpoints	31
4.27	Bulk operations	31
4.28	Response envelopes	31
4.29	Field validation	31
4.30	Pagination pitfalls	31
4.31	Search response shape	31
4.32	Actions and commands	32
4.33	Write semantics	32
4.34	Default limits	32
4.35	Authorization scopes	32
4.36	Client SDKs and examples	32
4.37	Backward-compatible changes	32
4.38	OpenAPI as a contract	32
4.39	Public versus internal APIs	33
4.40	Changelog and announcements	33
4.41	Deprecation headers	33
4.42	Error code taxonomy	33
4.43	Consistency across resources	33
4.44	Response examples	33
4.45	API review workflow	33
4.46	Design workshop	33
4.47	Contract testing loop	34
4.48	API governance	34
4.49	Client migration strategy	34
4.50	Field evolution playbook	34
4.51	API metrics	34
4.52	Client types	34
4.53	Discovery and navigation	34
4.54	Case study: mobile client constraints	35
4.55	SDK versioning	35
4.56	Error handling examples	35
4.57	Caching in API design	35
4.58	Case study: breaking change rollout	35
4.59	Compatibility testing example	35
4.60	Example resource representation	35
4.61	Review checklist	36
5	Data modeling	37
5.1	Entities and relationships	37
5.2	Constraints	37
5.3	Constraint examples	37
5.4	Indexes	37
5.5	Normalization and denormalization	37
5.6	Data ownership	38
5.7	Query patterns	38
5.8	Example query patterns	38
5.9	Migrations	38
5.10	Migration example	38
5.11	Soft deletes	38

5.12	Audit trails	38
5.13	Data retention	38
5.14	Backfills	38
5.15	Transactions and isolation	39
5.16	Read replicas	39
5.17	Explain plans	39
5.18	Integrity checks	39
5.19	Write amplification	39
5.20	Schema evolution	39
5.21	Partitioning	39
5.22	Backups and recovery	39
5.23	Data locality	39
5.24	Case study: activity log growth	39
5.25	Data validation at rest	40
5.26	Null handling	40
5.27	Data lifecycle and classification	40
5.28	Hot and cold data	40
5.29	Reference data	40
5.30	Sharding criteria	40
5.31	Event tables	40
5.32	Change data capture	40
5.33	Precision and types	40
5.34	Testing with realistic data	41
5.35	Hotspot avoidance	41
5.36	Index maintenance	41
5.37	Data fixes and runbooks	41
5.38	PII and compliance	41
5.39	Analytics schema	41
5.40	Time-series data	41
5.41	Foreign keys and deletes	41
5.42	Version columns	41
5.43	Multi-tenant schemas	42
5.44	Data quality monitoring	42
5.45	Archival strategy	42
5.46	Access patterns and tooling	42
5.47	Partitioning example	42
5.48	RPO and RTO	42
5.49	Materialized views	42
5.50	Data governance	42
5.51	Schema documentation	43
5.52	Backfill plan example	43
5.53	Migration phases	43
5.54	Validation queries	43
5.55	Sampling for checks	43
5.56	Indexing case study	43
5.57	Retention and purge jobs	43
5.58	Data drift monitoring	44
5.59	Schema example	44
5.60	Index and query example	44
5.61	Audit table example	44
5.62	Data dictionary example	44

6	Project structure	46
6.1	Layers	46
6.2	Example layout	46
6.3	Dependency direction	46
6.4	Boundaries and ownership	46
6.5	Package by feature	47
6.6	Public versus internal code	47
6.7	Dependency rules	47
6.8	Configuration layout	47
6.9	Migrations and data access	47
6.10	Testing layout	47
6.11	Example test layout	47
6.12	API boundaries	47
6.13	Shared libraries	48
6.14	Monorepo and multi-repo	48
6.15	Build and tooling	48
6.16	Documentation placement	48
6.17	Interfaces and ports	48
6.18	Wiring and composition	48
6.19	Configuration by environment	48
6.20	Module boundaries	48
6.21	Domain model placement	48
6.22	Example service interface	48
6.23	Example adapter	49
6.24	Runtime boundaries	49
6.25	Configuration for workers	49
6.26	Code ownership and reviews	49
6.27	Directory naming	49
6.28	Avoiding shared state	49
6.29	Dependency graph checks	49
6.30	Example module layout	49
6.31	Monorepo tooling	50
6.32	Release boundaries	50
6.33	What good structure optimizes for	50
6.34	Evolution stages	50
6.35	Bounded context mapping	50
6.36	Module size thresholds	50
6.37	API versioning layout	50
6.38	Cross-cutting concerns	50
6.39	Feature flags	51
6.40	Configuration hierarchy	51
6.41	Data contracts and schemas	51
6.42	Operational tooling	51
6.43	Deployment topology	51
6.44	Dependency inversion in practice	51
6.45	Decision logs	51
6.46	Onboarding path	51
6.47	Structural refactors	51
6.48	Enforcement tooling	52
6.49	Example dependency rule config	52
6.50	Case study: split a growing module	52
6.51	Ownership matrix	52

6.52	Example ownership table	52
6.53	README templates	52
6.54	Example module README	52
6.55	Versioned modules	53
6.56	Shared schema registry	53
6.57	Build targets	53
6.58	Testing pyramid alignment	53
6.59	Migration plan for structure changes	53
6.60	Anti-patterns	53
6.61	Pattern: facade modules	53
6.62	Pattern: replaceable adapters	53
6.63	Lifecycle of a new module	54
7	Validation and errors	55
7.1	Validation strategy	55
7.2	Error shape	55
7.3	Field errors	55
7.4	Mapping errors to status codes	55
7.5	Where validation lives	55
7.6	Schema validation	56
7.7	Partial update validation	56
7.8	Error taxonomy	56
7.9	Batch validation	56
7.10	Localization and messages	56
7.11	Logging and observability	56
7.12	Retry guidance	56
7.13	Example validation flow	57
7.14	Constraint violations	57
7.15	Consistency with logging	57
7.16	Validation for imports	57
7.17	Normalization	57
7.18	Reserved values	57
7.19	Error metadata	57
7.20	Example error response with metadata	57
7.21	Validation in async jobs	58
7.22	Client-side validation	58
7.23	Idempotency and validation	58
7.24	Error contracts	58
7.25	Validation as a product surface	58
7.26	Field path conventions	58
7.27	Partial update rules	58
7.28	Example partial update policy	58
7.29	Batch validation details	59
7.30	Error category semantics	59
7.31	Error category table	59
7.32	Error code ownership	59
7.33	Localization strategy	59
7.34	Normalization and preprocessing	59
7.35	Reserved values revisited	59
7.36	Constraint evolution	59
7.37	Async validation	59
7.38	Logging and privacy	60

7.39	Retries and idempotency	60
7.40	Example error envelope	60
7.41	Auditability	60
7.42	Testing validation	60
7.43	Validation architecture	60
7.44	Strict versus lenient modes	60
7.45	Backward compatibility	60
7.46	Error documentation	61
7.47	CSV import example	61
7.48	Example import error report	61
7.49	Constraint mapping	61
7.50	Example constraint mapping	61
7.51	Security validation	61
7.52	Field metadata	61
7.53	Error aggregation	61
7.54	Validation pipeline	61
7.55	Rate limit errors	62
7.56	Client UX guidance	62
7.57	State transition validation	62
7.58	Warning responses	62
7.59	Example warning response	62
7.60	Help links	62
7.61	Limits and caps	62
7.62	Contract tests	62
7.63	Numeric precision	63
7.64	Time zones	63
7.65	Schema versioning	63
7.66	Error quotas	63
7.67	Example error quota	63
8	Authentication and authorization	64
8.1	Authentication	64
8.2	Authorization	64
8.3	Resource ownership	64
8.4	Service-to-service auth	64
8.5	Token lifecycle	64
8.6	Session management	64
8.7	Claims and scopes	65
8.8	RBAC and ABAC	65
8.9	Resource checks	65
8.10	Service identity	65
8.11	Audit trails	65
8.12	Key rotation	65
8.13	Example policy table	65
8.14	Error responses	65
8.15	Password storage	65
8.16	MFA and step-up auth	66
8.17	Delegated access	66
8.18	API keys	66
8.19	Rate limiting	66
8.20	Policy evaluation	66
8.21	Admin actions	66

8.22	Service auth example	66
8.23	Secret storage	66
8.24	Threat model	66
8.25	Identity proofing	67
8.26	Account lifecycle	67
8.27	Password resets	67
8.28	Session binding	67
8.29	Token revocation	67
8.30	Scope design	67
8.31	Fine-grained checks	67
8.32	Policy composition	67
8.33	Authorization at the data layer	67
8.34	User impersonation	68
8.35	Audit log structure	68
8.36	Key management	68
8.37	Secret sprawl	68
8.38	Service identity onboarding	68
8.39	Security testing	68
8.40	Auth error responses	68
8.41	Example policy evaluation flow	68
8.42	Incident response	69
8.43	Single sign-on	69
8.44	Provisioning and deprovisioning	69
8.45	Access reviews	69
8.46	Device trust	69
8.47	Consent and transparency	69
8.48	Token format stability	69
8.49	JWT headers and key ids	69
8.50	mTLS for internal traffic	69
8.51	Authorization caching	69
8.52	Auth metrics	70
8.53	Policy drift	70
8.54	Example access review checklist	70
8.55	Tenant isolation	70
8.56	Account lockout	70
8.57	MFA recovery	70
8.58	Session limits	70
8.59	Policy as code	70
8.60	Credential stuffing defense	70
8.61	Example auth metrics	71
8.62	Tenant admin boundaries	71
8.63	SCIM provisioning	71
8.64	API access versus user access	71
8.65	Session timeout policy	71
8.66	Key rotation playbook	71
8.67	Break-glass access	71
8.68	Just-in-time access	71
8.69	Session anomaly detection	71
8.70	Security headers	72
8.71	Example conditional policy	72

9.1	Logs	73
9.2	Metrics	73
9.3	Tracing	73
9.4	Dashboards	73
9.5	Structured log example	73
9.6	RED and USE	74
9.7	SLIs and SLOs	74
9.8	Tracing context	74
9.9	Sampling	74
9.10	Alert design	74
9.11	Dashboard layers	74
9.12	Log retention	74
9.13	Incident timeline	74
9.14	Error budgets	75
9.15	Examples of SLIs	75
9.16	Instrumentation coverage	75
9.17	Metrics naming	75
9.18	Example metrics	75
9.19	Trace-to-log correlation	75
9.20	Golden signals	75
9.21	Alert fatigue	76
9.22	Runbooks	76
9.23	Log levels	76
9.24	Log hygiene	76
9.25	Correlation ids	76
9.26	Metrics types	76
9.27	Histogram buckets	76
9.28	Trace span design	76
9.29	Context propagation	76
9.30	Metrics cardinality	76
9.31	Exemplars	77
9.32	Synthetics	77
9.33	Real user monitoring	77
9.34	Queue and worker metrics	77
9.35	Dependency monitoring	77
9.36	Alert severity	77
9.37	On-call handoff	77
9.38	Incident review	77
9.39	Example alert rule	77
9.40	Observability maturity	78
9.41	Cost management	78
9.42	Time synchronization	78
9.43	Service maps	78
9.44	Health checks	78
9.45	SLO documentation	78
9.46	Example SLO document	78
9.47	Sampling strategy	78
9.48	Sampling table	79
9.49	Event logging	79
9.50	Data retention tiers	79
9.51	Privacy and compliance	79
9.52	Chaos drills	79

9.53	Observability reviews	79
9.54	Debugging workflow	79
9.55	Log sampling	79
9.56	Instrumentation backlog	79
9.57	Release gates	80
9.58	Alert tuning loop	80
9.59	Dashboard ownership	80
9.60	Burn rate alerts	80
9.61	Log search practices	80
9.62	Exemplar-driven debugging	80
9.63	Metric naming conventions	80
9.64	SLI calculation example	80
9.65	Error budget policy	80
9.66	Playbook template	81
10	Performance	82
10.1	Latency budgets	82
10.2	Profiling	82
10.3	Caching	82
10.4	Pagination and limits	82
10.5	Connection pooling	82
10.6	Query optimization	82
10.7	N+1 queries	83
10.8	Caching layers	83
10.9	Load testing	83
10.10	Capacity planning	83
10.11	Backpressure	83
10.12	Serialization costs	83
10.13	Performance regression tests	83
10.14	Example performance budget	83
10.15	Async offloading	83
10.16	Compression tradeoffs	84
10.17	Database tuning	84
10.18	Hot path optimization	84
10.19	Concurrency limits	84
10.20	Index maintenance	84
10.21	Read replicas	84
10.22	Performance debugging	84
10.23	Benchmarking	84
10.24	Case study: slow list endpoint	84
10.25	Throughput and latency	85
10.26	Tail latency drivers	85
10.27	Cache invalidation	85
10.28	Warmup behavior	85
10.29	Batching	85
10.30	Client timeouts	85
10.31	Circuit breakers	85
10.32	Queue sizing	85
10.33	Data model choices	85
10.34	Serialization formats	85
10.35	Memory pressure	86
10.36	Concurrency versus parallelism	86

10.37	Pagination strategy	86
10.38	Example cache strategy	86
10.39	Performance budgets per tier	86
10.40	Performance review checklist	86
10.41	Cache stampede	86
10.42	Critical path analysis	86
10.43	Async IO	87
10.44	Thread and worker pools	87
10.45	Load shedding strategies	87
10.46	Autoscaling policies	87
10.47	Index selectivity	87
10.48	Network latency	87
10.49	TLS overhead	87
10.50	Compression thresholds	87
10.51	Performance incident checklist	87
10.52	Example critical path	88
10.53	Database contention	88
10.54	Hot key mitigation	88
10.55	Write amplification	88
10.56	Profiling in production	88
10.57	Example load test plan	88
10.58	Slow query logging	88
10.59	Cache consistency	88
10.60	Sharding considerations	89
10.61	IO profiling	89
10.62	Memory leak detection	89
10.63	Latency regression triage	89
10.64	Cold cache benchmarking	89
10.65	Client-side caching	89
10.66	Lock contention profiling	89
10.67	SLA versus SLO	89
10.68	Connection pool tuning	89
10.69	Connection pool example	89
11	Background work	91
11.1	Queues and workers	91
11.2	Retries	91
11.3	Dead letter queues	91
11.4	Job idempotency	91
11.5	When to use background work	91
11.6	Queue types	91
11.7	At-least-once delivery	92
11.8	Visibility timeouts	92
11.9	Retry policy	92
11.10	Retry schedule example	92
11.11	Dead letter handling	92
11.12	Poison pill jobs	92
11.13	Job payload design	92
11.14	Priority queues	92
11.15	Scheduling and cron	92
11.16	Rate limiting jobs	93
11.17	Concurrency control	93

11.18	Ordering requirements	93
11.19	Idempotency keys	93
11.20	Example job record	93
11.21	Backpressure	93
11.22	Scaling workers	93
11.23	Job timeouts	93
11.24	Cancellation	93
11.25	Bulk processing	94
11.26	Observability for jobs	94
11.27	Example job metrics	94
11.28	Security considerations	94
11.29	Consistency with the request path	94
11.30	Job lifecycle	94
11.31	Leasing and locks	94
11.32	Exactly-once illusion	94
11.33	Deduplication storage	95
11.34	Payload versioning	95
11.35	Schema validation	95
11.36	Job state table	95
11.37	Fanout and chaining	95
11.38	Workflow orchestration	95
11.39	Backfills	95
11.40	Reprocessing policy	95
11.41	Priority inversion	95
11.42	Job sharding	96
11.43	Storage considerations	96
11.44	Operational tooling	96
11.45	Example job admin actions	96
11.46	Job SLAs	96
11.47	Job latency tracking	96
11.48	Multi-tenant fairness	96
11.49	Routing by job type	96
11.50	Blue/green for workers	96
11.51	Graceful shutdown	97
11.52	Poison pill workflow	97
11.53	Example job router	97
11.54	Backfill safety	97
11.55	Costs and limits	97
11.56	Side effects	97
11.57	Checkpointing	97
11.58	Batch windowing	97
11.59	Dedupe windows	98
11.60	Example dedupe record	98
11.61	Alerts for queues	98
11.62	Example queue alert	98
11.63	Job SLAs table	98
11.64	Payload size	98
11.65	Job testing	98
11.66	Schema registry	98
11.67	Reconciliation jobs	99
11.68	Outbox-driven jobs	99
11.69	Partitioning queues	99

11.70	Example partition config	99
12	Consistency	100
12.1	Transactions	100
12.2	Read-after-write	100
12.3	Eventual consistency	100
12.4	Outbox pattern	100
12.5	Consistency models	100
12.6	Isolation levels	100
12.7	Read your writes	101
12.8	Monotonic reads	101
12.9	Write skew	101
12.10	Quorum reads and writes	101
12.11	Replication lag	101
12.12	Cache consistency	101
12.13	Dual writes	101
12.14	Outbox processing	101
12.15	Conflict resolution	101
12.16	Versioning for writes	102
12.17	Example optimistic concurrency	102
12.18	Sagas	102
12.19	Idempotency and consistency	102
12.20	Consistency budget	102
12.21	Consistency examples	102
12.22	Testing consistency	102
12.23	CAP tradeoffs	102
12.24	Consistency and latency	103
12.25	Session guarantees	103
12.26	Anomaly table	103
12.27	Read repair	103
12.28	Quorum example	103
12.29	Multi-region consistency	103
12.30	Leader election	103
12.31	Clock skew	103
12.32	Idempotent events	103
12.33	Conflict example	104
12.34	Consistency communication	104
12.35	Causal consistency	104
12.36	Search indexes	104
12.37	Materialized views	104
12.38	Read models	104
12.39	Pagination consistency	104
12.40	Example outbox flow	104
12.41	Consistency in APIs	105
12.42	Write amplification	105
12.43	Linearizability	105
12.44	Causal ordering	105
12.45	Version vectors	105
12.46	Consistency in caches	105
12.47	Example cache policy	105
12.48	Two-phase commit	105
12.49	Sagas versus 2PC	105

12.50	Consistency guarantees table	106
12.51	Case study: search lag	106
12.52	Conflict resolution strategies	106
12.53	Conflict resolution table	106
12.54	Distributed locks	106
12.55	Freshness metrics	106
12.56	Example freshness metric	106
12.57	Case study: dual writes	106
12.58	Replication topologies	106
12.59	Lag-based routing	107
12.60	Consistency in batch jobs	107
12.61	Consistency testing strategies	107
12.62	Example replication config	107
12.63	Write fences	107
12.64	Client tokens	107
12.65	Consistency in pagination	107
12.66	Read barriers	107
12.67	Consistency in analytics	107
12.68	Staleness budgets	108
12.69	User-visible consistency	108
13	Reliability	109
13.1	Timeouts	109
13.2	Retries	109
13.3	Circuit breakers	109
13.4	Bulkheads	109
13.5	Graceful degradation	109
13.6	Error budgets	109
13.7	Health checks	109
13.8	Load shedding	110
13.9	Rate limiting	110
13.10	Fallbacks	110
13.11	Redundancy	110
13.12	Disaster recovery	110
13.13	Change management	110
13.14	Incident response	110
13.15	Post-incident reviews	110
13.16	Reliability metrics	110
13.17	Example reliability targets	111
13.18	Chaos testing	111
13.19	Capacity management	111
13.20	Example circuit breaker config	111
13.21	Timeout budgets	111
13.22	Retry budgets	111
13.23	Dependency isolation	111
13.24	Multi-region failover	111
13.25	Data durability	111
13.26	Service ownership	112
13.27	Monitoring maturity	112
13.28	Runbook example	112
13.29	Failure mode catalog	112
13.30	Reliability reviews	112

13.31	Dependency SLOs	112
13.32	Graceful degradation levels	112
13.33	Degradation table	112
13.34	Saturation signals	112
13.35	Progressive delivery	113
13.36	Synthetic monitoring	113
13.37	User-facing status	113
13.38	Example rollout policy	113
13.39	Hedged requests	113
13.40	Queue resilience	113
13.41	Feature flags	113
13.42	Operational readiness	113
13.43	Resilience scorecard	113
13.44	Example resilience scorecard	114
13.45	Alert quality	114
13.46	Client timeouts	114
13.47	Dependency mapping	114
13.48	Error budget policy	114
13.49	Example error budget policy	114
13.50	Incident communications	114
13.51	Example incident update	114
13.52	Failure modes	114
13.53	Failure mode table	115
13.54	Capacity headroom	115
13.55	Autoscaling policies	115
13.56	Traffic shaping	115
13.57	Postmortem template	115
13.58	SLO dashboards	115
13.59	Dependency budgets	115
13.60	Queue drain procedures	115
13.61	Resilient client behavior	116
13.62	Latency budgets for dependencies	116
13.63	Reliability guardrails	116
13.64	Backpressure	116
13.65	Cold start risk	116
13.66	Maintenance windows	116
13.67	Reliability debt	116
13.68	User impact mapping	116
13.69	Stateful service resilience	116
13.70	Escalation policy	116
13.71	Detection latency	117
13.72	Failure injection	117
13.73	Reliability and cost	117
13.74	On-call handoff	117
14	Security	118
14.1	Input handling	118
14.2	Secrets	118
14.3	Dependencies	118
14.4	Rate limiting	118
14.5	Threat modeling	118
14.6	Least privilege	118

14.7 Encryption in transit	119
14.8 Encryption at rest	119
14.9 Security headers	119
14.10Dependency scanning	119
14.11Secrets management	119
14.12Audit logging	119
14.13Data classification	119
14.14Common attack vectors	119
14.15Example security controls	120
14.16Security testing	120
14.17Incident response	120
14.18Key rotation	120
14.19Secure defaults	120
14.20Authentication hardening	120
14.21Authorization boundaries	120
14.22Input validation	120
14.23Secure logging	120
14.24Data retention and deletion	121
14.25Security reviews	121
14.26Example security checklist	121
14.27Secure SDLC	121
14.28Supply chain security	121
14.29Container and host security	121
14.30Network segmentation	121
14.31WAF and edge protections	121
14.32Secrets rotation plan	121
14.33Example secret rotation	122
14.34Data access auditing	122
14.35CSRF protection	122
14.36SSRF protection	122
14.37Input encoding	122
14.38Vulnerability management	122
14.39Incident response runbook	122
14.40Security monitoring	122
14.41Data classification table	123
14.42IAM policy example	123
14.43Data minimization	123
14.44Secure deletion	123
14.45Security training	123
14.46Secure configuration	123
14.47Access reviews	123
14.48Security metrics	123
14.49Example security event	124
14.50Data residency	124
14.51Secure backups	124
14.52Key management	124
14.53Security reviews for changes	124
14.54Configuration drift	124
14.55Secure defaults checklist	124
14.56Secrets sprawl	124
14.57Security posture reviews	124
14.58Patch cadence	125

14.59	Session management	125
14.60	Password storage	125
14.61	Token handling	125
14.62	File upload security	125
14.63	CORS policy	125
14.64	Third-party risk	125
14.65	Secrets in CI	125
14.66	Security champions	125
14.67	Security SLAs	125
14.68	Audit readiness	126
15	Testing	127
15.1	Unit tests	127
15.2	Integration tests	127
15.3	Contract tests	127
15.4	Test data	127
15.5	Test pyramid	127
15.6	End-to-end tests	127
15.7	Property-based testing	127
15.8	Fuzz testing	128
15.9	Mutation testing	128
15.10	Flaky tests	128
15.11	Test environments	128
15.12	CI pipelines	128
15.13	Test data management	128
15.14	Contract testing	128
15.15	Mocking strategy	128
15.16	Performance testing	128
15.17	Canary testing	128
15.18	Example test matrix	129
15.19	Example test config	129
15.20	Snapshot testing	129
15.21	Database migration tests	129
15.22	Test isolation	129
15.23	Test coverage	129
15.24	Data factories	129
15.25	Golden path testing	129
15.26	Security testing	129
15.27	Test reports	130
15.28	Static analysis	130
15.29	Test selection	130
15.30	Parallelization	130
15.31	Release testing	130
15.32	Example CI pipeline	130
15.33	Test ownership	130
15.34	Test strategy document	130
15.35	Test debt	130
15.36	Flaky test triage	131
15.37	Non-functional testing	131
15.38	Contract evolution	131
15.39	Example test strategy outline	131
15.40	Environment parity	131

15.41	Test data seeding	131
15.42	Test containers	131
15.43	Test stability metrics	131
15.44	Example stability metric	131
15.45	Testing in production	132
15.46	Contract test example	132
15.47	Test naming	132
15.48	Test refactoring	132
15.49	Observability for tests	132
15.50	Test environments table	132
15.51	Data migration testing	132
15.52	Performance regression tests	132
15.53	Chaos testing in staging	133
15.54	Test suite hygiene	133
15.55	Test data cleanup	133
15.56	Test performance budgets	133
15.57	Example test budget	133
15.58	Test prioritization	133
15.59	Flake quarantine	133
15.60	Boundary testing	133
15.61	Concurrency testing	133
15.62	Clock control	134
15.63	Test doubles	134
15.64	Test doubles table	134
15.65	Schema testing	134
15.66	Replay testing	134
15.67	Data privacy in tests	134
15.68	Test review	134
15.69	Risk-based testing	134
15.70	Example risk matrix	134
15.71	API testing	135
15.72	Queue processing tests	135
15.73	Feature flag testing	135
15.74	Rollback testing	135
15.75	Load test fixtures	135
15.76	Test tagging	135
15.77	Example test tags	135
15.78	Schema migration rehearsal	135
15.79	Eventual consistency tests	135
16	Deployments	137
16.1	Deployment strategy	137
16.2	Migrations	137
16.3	Rollback	137
16.4	Configuration	137
17	Architecture choices	138
17.1	Monoliths	138
17.2	Microservices	138
17.3	Modular monoliths	138
17.4	Decision criteria	138

18 Event-driven systems	139
18.1 Events and commands	139
18.2 Schemas	139
18.3 Ordering and duplication	139
18.4 Backpressure	139
19 Multi-tenancy	140
19.1 Isolation models	140
19.2 Tenant identification	140
19.3 Data access rules	140
19.4 Noisy neighbors	140
20 Production readiness	141
20.1 Runbooks	141
20.2 Alerts	141
20.3 Capacity	141
20.4 On-call	141
A Appendix: checklists	142
A.1 API design checklist	142
A.2 Data modeling checklist	142
A.3 Operations checklist	142
A.4 Security checklist	142
B Appendix: templates	143
B.1 Standard error response	143
B.2 Pagination response	143
B.3 SQL migration template	143
B.4 Health check endpoint	143

Preface

This book is about building backends that are easy to change and safe to operate. It is written for engineers who want to ship services and keep them healthy in production.

The approach is opinionated:

- keep business logic out of frameworks
- design APIs for change
- treat data as a first-class interface
- measure what matters (latency, errors, saturation)
- keep security and reliability in the default path

Chapter 1

Introduction

Backend development is the work of building systems that other systems depend on. A backend receives requests, validates them, applies business rules, changes state, and returns results.

Most teams do not fail because they cannot write endpoints. They fail because the backend becomes hard to change. This happens when interfaces drift, data models become inconsistent, and operational reality is ignored.

1.1 A reference system

Throughout the book, assume a simple product: a **Task API** that supports projects, tasks, and activity logs. It is enough to discuss real constraints: authentication, pagination, background work, indexing, and deployments.

1.2 What “good” looks like

A backend is in good shape when:

- changes are local and predictable
- failures are visible and recoverable
- performance is measured, not guessed
- security does not rely on developer memory
- operations are boring

1.3 Scope and non-goals

This book focuses on production systems: APIs, data, and operations. It does not cover UI concerns or low-level kernel tuning. If you keep the system simple and observable, most hard problems become manageable.

1.4 A minimal API surface

A reasonable starting API for the Task system:

- GET /projects
- POST /projects
- GET /projects/:id
- POST /projects/:id/tasks

- PATCH /tasks/:id
- GET /tasks?project=:id

1.5 Quality attributes

Backends are judged by qualities, not just features. These attributes are the baseline for production systems.

Attribute	What it means in practice
Reliability	Requests succeed or fail predictably, with safe retries
Performance	Latency stays within a known budget at peak load
Security	Access is controlled, secrets are protected, audit trails exist
Scalability	Capacity grows without redesigning the entire system
Maintainability	Changes are small, testable, and reversible

1.6 The operating model

Every backend is an operations system. If you cannot deploy, observe, and recover safely, features do not matter. Make operational tasks part of the design:

- define health checks and readiness signals
- track error budgets and latency budgets
- build runbooks for common failures

1.7 A small data model

The Task system is intentionally small:

- **Project:** id, name, owner
- **Task:** id, project id, title, status, due date
- **Activity:** id, task id, action, timestamp

This is enough to discuss ownership, filtering, pagination, and background work.

1.8 Change as the primary constraint

The hardest part of backend work is change. You will add fields, remove endpoints, move data, and split services. A good backend is one where these changes are possible without large rewrites.

Pattern: treat interfaces as products

The API is not a technical detail. It is a contract with clients, future developers, and your operations team.

1.9 Common failure modes

Teams often fall into the same traps:

- business logic hidden in controllers and frameworks
- inconsistent error formats across endpoints
- databases without constraints or indexes
- no measurable latency or error budgets
- ad-hoc security rules

1.10 Working style

This book assumes you work in small, focused changes:

- keep endpoints small and composable
- move complex logic into services
- test the critical path first
- monitor before you optimize

1.11 Who this is for

This is for engineers who ship and operate backends:

- product teams running APIs in production
- platform teams enabling internal services
- engineers migrating from prototypes to durable systems

1.12 System boundaries

Backends are not just servers. They include storage, queues, caches, and the tooling around them. Draw a boundary around what you own and what you depend on. Every dependency outside that boundary needs monitoring and a fallback plan.

1.13 A sample request

A common request in the Task API is creating a task. The response is intentionally simple:

```
{
  "id": "task_123",
  "project_id": "proj_7",
  "title": "Write docs",
  "status": "open",
  "created_at": "2026-02-18T10:00:00Z"
}
```

1.14 Operational baseline

Every backend should define a baseline operational posture:

- health checks and readiness endpoints
- log correlation identifiers
- a dashboard for latency, errors, and traffic
- alerts tied to user-visible symptoms

1.15 Documentation artifacts

Keep lightweight documentation close to the code:

- an API reference or OpenAPI spec
- runbooks for the top three incidents
- a short architecture overview for new engineers

1.16 What a backend is responsible for

Backends are responsible for more than serving HTTP:

- enforcing business rules consistently
- safeguarding data integrity
- managing concurrency and coordination
- providing stable contracts to clients
- operating safely under load

1.17 How backends fail in practice

Backends usually fail slowly before they fail completely. Latency creeps up, error rates climb, and on-call becomes reactive. If you monitor latency, errors, and saturation, you can intervene early.

1.18 A short architecture sketch

The Task API can be implemented with a simple architecture:

- an API service for HTTP requests
- a relational database for tasks and projects
- a queue for activity events and background jobs
- a cache for hot reads

1.19 Development lifecycle

Production systems require a lifecycle:

- local development with realistic data
- staged environments for integration tests
- production deploys with rollback paths
- post-deploy monitoring and validation

1.20 Tradeoffs are expected

There is no perfect design. You choose tradeoffs based on expected load, team size, and product risk. Make those tradeoffs explicit and document them.

1.21 From prototype to product

Most backends start as a prototype. The difference between prototype and product is the ability to change safely. Add constraints, monitoring, and tests before you add more endpoints.

1.22 Operational constraints

Production systems operate under constraints:

- limited error budgets
- fixed database capacity
- on-call fatigue
- unpredictable traffic patterns

1.23 Interfaces beyond HTTP

Your backend interfaces include more than HTTP endpoints. Database schemas, queues, and background job payloads are also contracts. Treat those contracts with the same care as API endpoints.

1.24 Release quality gates

Define a minimal set of gates before each deploy:

- unit and integration tests
- database migration checks
- smoke tests on staging
- rollback validation

1.25 Who depends on you

Clients are not just user interfaces. They include other services, data pipelines, and internal tooling. The more dependents you have, the more you need stable interfaces.

1.26 Reference workflow

A minimal workflow for backend changes:

- write or update the API contract
- implement the change in the service layer
- add tests for the critical path
- update migrations and run locally
- deploy to staging and validate metrics

1.27 Baseline metrics

Start with a small, stable set of metrics:

- request rate per endpoint
- error rate per endpoint
- latency p50, p95, p99
- database query latency
- queue depth for background work

1.28 Developer experience

If the development loop is slow, engineers will skip tests and avoid refactors. Invest in a fast local setup, representative test data, and deterministic tooling.

1.29 Case study: growth without chaos

The Task API starts with a single service and a single database. As usage grows, new requirements appear: search, analytics, and integrations. If the API contract and data model are stable, these changes are incremental. If they are ad-hoc, every new feature becomes a migration project.

The lesson is simple: design for change early. The first version does not need to be perfect, but it must be consistent.

1.30 Configuration and environments

Backends live in multiple environments: local, staging, and production. Configuration must be external to code and validated on startup. If a service cannot start without configuration, fail fast and clearly.

1.31 Feature flags

Feature flags are a safe way to roll out risky changes. They allow you to test in production with a small subset of traffic. Use flags sparingly and remove them after the rollout.

1.32 Data contracts

Data is a contract just like the API. When you add or change a field, you are changing the contract. Document those changes and plan migrations explicitly.

1.33 Incident response

Expect incidents and rehearse your response. A good incident response includes:

- clear ownership and on-call rotation
- runbooks for top failure modes
- post-incident reviews with action items

1.34 Scaling the team

As teams grow, communication becomes the bottleneck. Stable interfaces, clear ownership, and shared standards keep teams aligned.

1.35 Operational posture

A stable backend has a known operational posture. That posture includes how you deploy, how you observe, and how you recover. If these are undocumented, the system will only be safe in the heads of a few engineers.

1.36 Production constraints you cannot ignore

Every production system has constraints that shape design:

- limited database connections
- external API rate limits
- on-call hours and handoff processes
- compliance and audit requirements

1.37 API evolution mindset

Assume the API will evolve continuously. Design so that adding fields and new endpoints is routine. Treat breaking changes as exceptional events with a clear process.

1.38 Example ownership map

Ownership keeps change safe. A simple ownership map for the Task system might be:

- Projects: core backend team
- Tasks: core backend team
- Activity logs: platform team
- Search and analytics: data team

1.39 Release reliability

Reliability is not just runtime behavior. It includes the release process: how you migrate data, roll out changes, and rollback. If releases are unsafe, production will be unsafe too.

1.40 Assumptions and load profile

Every backend makes assumptions about scale and shape of traffic. Write them down, even if they are rough:

- expected requests per second at peak
- typical request sizes and response sizes
- daily batch windows or traffic spikes
- read-to-write ratio for core endpoints

These assumptions guide indexing, caching, and capacity planning. When they change, the system needs to change with them.

1.41 Security baseline

Security is part of the contract. A minimal baseline includes:

- explicit authentication and authorization checks
- secrets managed outside the codebase
- encrypted transport and sensitive data handling
- an audit trail for sensitive operations

If these are implicit, they will be inconsistent.

1.42 Cost awareness

Cost is a constraint like latency and availability. Small decisions compound into large bills. Start with a simple discipline:

- know the cost drivers (database, queue, storage, egress)
- track usage at the service level
- set budgets for non-critical workloads

1.43 Dependency management

Every backend depends on other systems. Make those dependencies explicit:

- document upstream and downstream services
- define timeouts and retry policies per dependency
- record the expected availability and failure modes

1.44 Architecture decisions

Some choices are durable: database, messaging system, schema strategy. Capture them in short Architecture Decision Records. The goal is not documentation volume, but traceability: future engineers should see why a decision was made.

1.45 Readiness review

Before calling a system production-ready, ask:

- Can we deploy without downtime?
- Can we restore data from backup?
- Do we know our p95 latency for key endpoints?
- Can we detect a silent failure within minutes?

1.46 Operational drills

Backups are only real after a restore. Run at least one restore drill per quarter and record the outcome. Treat drills as a feature: a backend that cannot be restored is incomplete.

1.47 Service catalog

As your system grows, keep a service catalog. It should list owners, on-call rotations, dependencies, and critical endpoints. This prevents knowledge from living only in people's heads.

1.48 Risk register

Every backend has known risks. Capture the top three risks and update them as the system changes:

- single database instance
- weak observability for background jobs
- ad-hoc access to production data

1.49 Definition of done

Feature work is not done until it is safe to operate. A minimal definition of done includes:

- tests for the critical path
- metrics and dashboards updated
- runbook notes for new failure modes

1.50 When to split services

Splitting too early adds complexity. Split when a domain is stable, owned by another team, and scaling needs are clear. If those conditions are not met, a well-structured modular monolith is safer.

1.51 Operating principles

Good backends follow a small set of principles:

- keep the critical path short
- prefer explicit contracts over implicit behavior
- make failures visible before they become outages
- optimize for reversible change

If a design violates these principles, it will be expensive to operate later.

1.52 Stakeholder alignment

Backend work impacts product, data, and operations. Align early on:

- data ownership and access policies
- SLA expectations from clients and partners
- maintenance windows and on-call coverage

Alignment is not paperwork; it prevents future incidents.

1.53 Example release plan

A simple release plan for a risky change:

- ship a backward-compatible schema change
- deploy code that reads old and new data
- backfill existing records
- enable the new behavior behind a feature flag
- remove the old field after a full rollout

Checklist: starting point

- Define the core resources and their identifiers.
- Decide how errors will be represented.
- Write the simplest endpoints that cover the use cases.
- Create a baseline dashboard for latency and errors.

Chapter 2

The backend mental model

Backends are systems, not just code. A request enters the system, touches data and dependencies, and returns a response. If you can describe that flow, you can reason about correctness and performance.

2.1 Request lifecycle

A typical request passes through:

- authentication and authorization
- validation and parsing
- business logic
- data access
- response shaping

2.2 A concrete flow

Consider `POST /projects/:id/tasks`. A clean flow looks like this:

- authenticate the user
- validate the input payload
- check project membership
- write the task in a transaction
- emit an activity event
- return the task

2.3 State and side effects

State changes should be explicit and traceable. Prefer to write state once per request, and keep side effects idempotent. If you need multiple writes, use a transaction or an outbox.

2.4 Dependencies

Most production bugs come from dependencies: databases, caches, queues, external APIs. Treat each dependency as a failure domain with its own latency and error modes.

Pattern: map the critical path

List every dependency touched by a request. That list is the true performance budget.

2.5 Failure modes

Every request can fail due to:

- invalid input
- missing data
- timeouts
- partial failures in dependencies

2.6 Backpressure

A backend must protect itself under load. Use queues, rate limits, and admission control to avoid collapse. If you accept all traffic, you risk a full outage.

2.7 Concurrency

Concurrency is where many bugs hide. Decide where you allow concurrent writes and how you resolve conflicts. Use unique constraints and transactions to make those decisions explicit.

2.8 A dependency budget

Every dependency consumes part of your latency budget. If the total exceeds the budget, you will miss SLOs regardless of code quality.

Dependency	Purpose	Budget (ms)
Primary database	Read/write project and task data	40
Cache	Read hot project metadata	5
Queue	Enqueue activity log event	10
External auth	Token validation	30

2.9 State machines

Many backend entities are state machines. Make the allowed transitions explicit and validate them on write. For tasks, a simple state machine might be:

- open → in-progress
- in-progress → done
- done → archived

2.10 Idempotency

Retried requests are normal in distributed systems. Use idempotency keys for create operations and store the result for a short period.

2.11 Idempotency key example

```
{
  "key": "req_42",
  "resource_id": "task_123",
  "stored_at": "2026-02-18T10:00:00Z"
}
```

2.12 Timeouts and deadlines

Set timeouts on outbound calls and use a request deadline budget. If a request has 200 ms, do not allow any dependency to consume more than its share.

2.13 Synchronous vs asynchronous work

Do synchronous work only when the user needs it immediately. Move expensive work into background jobs and return a job id. This keeps the request path fast and predictable.

2.14 Data consistency boundaries

Define which reads must be strongly consistent and which can be eventually consistent. The boundary should be based on user expectations, not on convenience.

2.15 Failure taxonomy

Classify failures so you can respond consistently:

- user errors (bad input, missing permissions)
- dependency errors (timeouts, 5xx from upstreams)
- internal errors (bugs, invariants broken)

2.16 Recoverability

If a dependency fails, decide whether to fail the request or degrade gracefully. Write down fallback behavior in code and in runbooks.

2.17 Load and saturation

When systems saturate, latency spikes and queues grow. Detect saturation early with queue depth, connection pool usage, and CPU load.

2.18 Cache strategy

Caches are part of the system model. Decide which reads are cacheable, how long entries live, and how to invalidate them. Stale cache data is a correctness risk, not just a performance issue.

2.19 Command and query separation

Writes are commands and reads are queries. Keep the write path strict and validated; keep the read path optimized and predictable.

2.20 Failure budget example

If your SLO is 99.9 percent, you have about 43 minutes of error budget per month. Spend that budget intentionally and avoid burning it on avoidable failures.

Warning: hidden dependencies

Any dependency not in your critical path list will surprise you during incidents. Make the dependency list explicit and keep it updated.

2.21 Deployment as part of the model

Deployments are another source of change. Design so that old and new versions can coexist during a rollout.

2.22 Read path vs write path

The read path is usually optimized for latency. The write path is optimized for correctness and durability. Do not treat them as identical; they have different constraints.

2.23 Invariants

Invariants are rules that must always hold. For example: a task must belong to exactly one project. Enforce invariants in the database and in the core domain model.

2.24 Race conditions

Two requests can arrive at the same time and both appear valid. Use unique constraints or compare-and-swap updates to prevent double writes.

2.25 Optimistic locking

Optimistic locking prevents lost updates without heavy locks. Store a version field and reject updates that use stale versions.

```
UPDATE tasks
SET title = 'New title', version = version + 1
WHERE id = 'task_123' AND version = 7;
```

2.26 Critical path example

For POST `/projects/:id/tasks`, a critical path might be:

- authenticate user (external auth)
- validate payload (local)
- write task (database)
- enqueue activity (queue)

2.27 Outbox pattern

If you need to publish an event after a transaction, use an outbox table. Write the event in the same transaction and publish it asynchronously.

2.28 Sagas and orchestration

Cross-service workflows require orchestration. Use sagas to coordinate multiple steps and compensate on failure.

2.29 Graceful degradation

Not all failures should cause full errors. If the activity queue is down, you may still want to create the task and log a warning.

2.30 Observability targets

Instrument at the edge and at dependency boundaries. You need to see total latency, dependency latency, and error rates separately.

2.31 Request timeline

A useful mental model is a timeline of a request:

- 0–10 ms: auth and validation
- 10–60 ms: database reads and writes
- 60–80 ms: cache or queue writes
- 80–100 ms: response building

2.32 Consistency ladder

Not all reads need the same guarantees. Define a ladder from strong consistency to eventual consistency and document it.

2.33 Coordination patterns

When you must coordinate across services, choose a pattern:

- synchronous calls for tight coupling
- asynchronous events for loose coupling
- sagas for multi-step workflows

2.34 Replay and recovery

If a message or event is lost, you need a way to replay it. Store durable logs or build replay endpoints for critical workflows.

2.35 Observability as feedback

Observability turns unknown failures into known issues. If you do not measure, you will regress silently.

2.36 Incident example

A slow dependency can turn into a system outage. Imagine external auth latency rising from 30 ms to 500 ms. If you have no timeout, every request queues until the service collapses. With timeouts and backpressure, you degrade gracefully and recover quickly.

2.37 SLOs and error budgets

SLOs translate reliability into a number. Error budgets help teams decide when to ship features versus stability work.

2.38 State and isolation

Each request should be safe to retry. If a request is not idempotent, protect it with keys or unique constraints.

2.39 Queueing and asynchronous guarantees

Queues change the model: a request may succeed while work continues later. Define the guarantees explicitly:

- at-least-once delivery for activity logs
- ordering within a single task or project
- maximum delay before processing

If you cannot state the guarantee, clients cannot reason about correctness.

2.40 Timeouts, retries, and budgets

Retries without bounds amplify failure. Tie retries to a budget and only retry when it is safe.

Layer	Retry policy	Max attempts
Client → API	retry on 429/503 with jitter	2
API → database	no retries, fail fast	1
API → external auth	retry on timeout	2

2.41 Burst handling

Most backends see bursts, not smooth traffic. Use a combination of rate limits, queues, and autoscaling. The goal is to keep the critical path stable while the backlog absorbs spikes.

2.42 Freshness and staleness

Some data can be stale without harming the user. Explicitly decide freshness targets:

- project metadata: 5 minutes
- task list: 30 seconds
- activity log: eventually consistent

2.43 Multi-tenant boundaries

If the system is multi-tenant, isolation is part of the model. Define how you prevent cross-tenant reads, writes, and noisy-neighbor effects. This drives indexing, rate limits, and partitioning choices.

2.44 Batch versus online

Batch jobs can consume the same data as online APIs. Keep them separate in the model so batch work does not starve online traffic. Schedule heavy batch work during low-traffic windows and cap its throughput.

2.45 Traffic classes

Not all traffic is equal. Separate traffic into classes:

- interactive requests with tight latency targets
- background processing with looser deadlines
- internal maintenance and backfills

This model informs priority queues and rate limits.

2.46 Backpressure signals

Backpressure is more than rate limiting. Expose explicit signals when the system is saturated.

Signal	Meaning
HTTP 429	client should retry with backoff
HTTP 503	service is overloaded or dependency is down
Queue depth	downstream processing is lagging

2.47 Jittered retries

When clients retry at the same time, spikes get worse. Add jitter to retries so they spread out. This small change can prevent cascading failures.

2.48 Request cancellation

If a client disconnects, the server should stop doing work. Support cancellation in long-running handlers and background jobs. This prevents wasted compute and reduces tail latency.

2.49 Single-writer principle

When possible, enforce a single writer per entity. This reduces contention and simplifies reasoning about state. If multiple writers are necessary, centralize conflict resolution.

2.50 Latency budget worked example

Assume a 200 ms p95 target for `POST /projects/:id/tasks`. Break the budget down explicitly and hold each component accountable.

Component	Notes	Budget (ms)
Auth check	local cache plus token validation	20
Validation	JSON parsing and schema checks	10
Database write	insert task and update counters	60
Queue publish	enqueue activity event	20
Response	render and serialize JSON	15
Buffer	headroom for variability	75

If a component consistently exceeds its budget, the whole SLO fails.

2.51 Tracing as a model

Tracing is not just for debugging. It validates your mental model by showing where time is spent. Use traces to verify critical path assumptions and identify hidden dependencies.

2.52 Capacity planning

Plan capacity with a simple envelope:

- peak requests per second
- expected data growth per month
- worst-case dependency latency

This does not need to be perfect, but it must be explicit.

2.53 Dependency matrix

Make a small matrix of which endpoints call which dependencies. This highlights the blast radius of a failing service.

Endpoint	Database	Cache	External auth
GET /projects	read	read	validate
POST /projects	write	none	validate
POST /projects/:id/tasks	write	read	validate

2.54 Queue sizing

Queues hide latency but do not remove it. Estimate queue growth during peak traffic and validate that consumers can catch up. If the queue does not drain within a business day, you will create operational debt.

2.55 Retry budgets per dependency

Not every dependency deserves retries. Define a retry budget per dependency to avoid runaway load. If a dependency is down, fail fast and degrade gracefully.

2.56 Case study: timeout cascade

Imagine the database latency doubles during peak traffic. Without timeouts, every request piles up and the thread pool saturates. With timeouts and queue limits, the system sheds load and recovers.

The mental model here is simple: you must protect the core dependency. Slow is not the same as down, but it can be more dangerous.

2.57 Saturation checklist

When latency rises, check the saturation signals:

- request queue depth
- database connection pool usage
- CPU and memory pressure
- dependency timeouts and retries

Checklist: mental model

- Know the request lifecycle.
- Identify dependencies and their failure modes.
- Make state changes explicit and reversible.
- Add backpressure before you need it.

Chapter 3

HTTP that matters

HTTP is the protocol your backend speaks. You do not need every feature, but you need the fundamentals.

3.1 Methods and semantics

Use methods for intent:

- GET for reads
- POST for creation
- PATCH for partial updates
- DELETE for removal

3.2 Status codes

Status codes are signals to clients and monitoring. Use them consistently.

- 200 success
- 201 created
- 400 validation error
- 401 unauthenticated
- 403 unauthorized
- 404 not found
- 409 conflict
- 500 unexpected error

3.3 Status code mapping

Write down a mapping table. Keep it consistent across endpoints:

Code	Typical meaning
200	Successful read or update
201	Resource created successfully
202	Accepted for background processing
204	Successful response with no body
400	Validation or parsing error
401	Authentication missing or invalid
403	Authenticated but not allowed
404	Resource not found
409	Conflict with existing data
429	Rate limit exceeded
500	Unhandled server error

3.4 Idempotency

Writes should be safe to retry. Use idempotency keys for payment and job creation endpoints.

3.5 Headers that matter

Headers carry important semantics:

- **Content-Type** for payload format
- **Accept** for response negotiation
- **Authorization** for authentication
- **Idempotency-Key** for safe retries

3.6 Idempotency in practice

Store the request hash and response for a short period. If the same key appears again, return the stored response.

```
{
  "idempotency_key": "req_9c6",
  "status": "stored",
  "expires_in_seconds": 86400
}
```

3.7 Pagination

Never return unbounded lists. Choose cursor-based pagination for large datasets.

```
{
  "items": [
    {"id": "task_123", "title": "Write docs"}
  ],
  "next_cursor": "task_123"
}
```

3.8 Caching headers

Set caching headers for read-heavy endpoints. Use **ETag** or **Last-Modified** to reduce bandwidth.

3.9 Content negotiation

Support at least JSON and make it explicit. Reject unexpected content types with a clear error.

3.10 Safe and unsafe methods

Clients and proxies treat methods differently. Ensure `GET` is safe and idempotent, and use `POST` for non-idempotent writes.

3.11 Streaming responses

If results are large, consider streaming or chunked responses. This keeps memory usage stable and improves time to first byte.

3.12 Request size limits

Set request size limits to protect memory and avoid abuse. Document those limits for clients.

3.13 Problem details

Consider using a standard error envelope or problem details format. Consistency is more important than the specific shape.

3.14 Error response example

```
{
  "error": {
    "code": "validation_error",
    "message": "title is required",
    "field": "title"
  }
}
```

3.15 Request body size

Set a maximum request body size. Large bodies can exhaust memory and slow downstream systems.

3.16 Client retry behavior

Document which endpoints are safe to retry. If a client retries non-idempotent requests, you will get duplicate data.

3.17 Caching strategy

Use `Cache-Control` to define shared and private caching. If responses contain user-specific data, avoid shared caches.

3.18 Content type versioning

If you need versioning, consider vendor media types. It is more explicit than path-based versioning and does not change URLs.

3.19 Conditional requests

Use conditional requests to avoid sending full responses when data has not changed. If the client presents an `If-None-Match` value and the resource is unchanged, return `304 Not Modified`.

3.20 ETag example

```
{
  "etag": "task_123:v7",
  "last_modified": "2026-02-18T10:00:00Z"
}
```

3.21 Rate limiting

Expose rate limits via headers so clients can behave responsibly:

- `X-RateLimit-Limit`
- `X-RateLimit-Remaining`
- `X-RateLimit-Reset`

3.22 Retry-After

When returning `429` or `503`, include `Retry-After`. This helps clients back off and prevents thundering herds.

3.23 Timeouts

Clients should not wait forever. Publish expected timeouts for critical endpoints and enforce them on the server.

3.24 Compression

Enable `gzip` or `brotli` for large responses. Compression reduces bandwidth and often improves tail latency.

3.25 Correlation identifiers

Include a request id in every response and log. This enables fast incident debugging and request tracing.

3.26 Example request

```
curl -i https://api.example.com/tasks \
  -H "Authorization: Bearer <token>" \
  -H "Accept: application/json"
```

3.27 Example response

```
{
  "items": [
    {"id": "task_123", "title": "Write docs"}
  ],
  "next_cursor": "task_123"
}
```

3.28 Health and readiness endpoints

Expose simple health checks for load balancers and deployment systems. Keep them fast and side-effect free.

3.29 Batch endpoints

Batch endpoints can reduce request overhead but increase complexity. Use them only when the latency or cost savings are meaningful.

3.30 CORS

If browsers call your API directly, configure CORS explicitly. Avoid broad wildcards for production systems.

3.31 Security headers

Security headers like `Strict-Transport-Security` and `X-Content-Type-Options` reduce attack surface for browser-based clients.

3.32 Authentication patterns

Be explicit about how clients authenticate:

- short-lived bearer tokens for user-facing clients
- long-lived API keys for server-to-server calls
- mTLS for highly sensitive internal services

Each pattern implies different rotation and monitoring requirements.

3.33 Authorization errors

Distinguish between unauthenticated and unauthorized. Return 401 when authentication is missing or invalid, and 403 when the user is authenticated but not allowed. This improves client behavior and audit clarity.

3.34 Request validation

Validate at the edge and return structured errors. Reject unknown fields if you want a strict contract. If you allow unknown fields, document that explicitly and ignore them safely.

3.35 Partial responses

Large responses can be expensive to produce and transmit. Allow clients to request a subset of fields, and default to a safe, stable set. This keeps the API responsive as resources grow.

3.36 Webhooks and callbacks

Webhooks are part of the HTTP surface area. Treat them as public endpoints:

- sign webhook payloads
- include a request id for retries
- document the retry strategy and time window

3.37 Long-running work

If work takes more than a few seconds, return 202 `Accepted` with a job id. Expose a `GET /jobs/:id` endpoint so clients can poll.

3.38 Error budget for clients

Client behavior is part of reliability. Publish clear guidance:

- which endpoints are safe to retry
- the maximum recommended retry window
- how to detect throttling

3.39 Input size and pagination limits

Define maximum page size and request body size. When a client exceeds limits, respond with 400 and explain the constraint.

3.40 Conditional request example

Clients can avoid full responses using conditional requests:

```
curl -i https://api.example.com/tasks/task_123 \  
-H "If-None-Match: \"task_123:v7\""
```

If the resource is unchanged, the server returns 304 with no body.

3.41 ETag and If-Match for writes

Use `If-Match` to prevent lost updates. If a client sends a stale ETag, return 412 `Precondition Failed`. This is a clean way to protect concurrent updates.

3.42 File uploads

If you accept files, document limits and formats. Large uploads should use multipart uploads or pre-signed URLs to keep the API responsive.

3.43 HTTP caching workflow

For read-heavy endpoints, caching should be part of the workflow:

- server returns `ETag` and `Cache-Control`
- client revalidates with `If-None-Match`
- server returns 304 if unchanged

This reduces bandwidth and avoids unnecessary processing.

3.44 Problem details with metadata

If you use a problem details format, include a stable error code and a trace id. This makes it easier to correlate client failures with server logs.

3.45 Header budget

Headers are not free. Large headers increase latency and can exceed gateway limits. Keep custom headers minimal and document those that matter.

3.46 Response shape stability

Clients depend on response shape. Avoid changing data types or omitting fields for existing endpoints. If you need a new representation, introduce a new field or a new endpoint.

3.47 Cache-Control directives

Make caching explicit with `Cache-Control`:

- `max-age` for browser and client caches
- `s-maxage` for shared caches
- `private` for user-specific responses

3.48 HTTP response example

An explicit response makes behavior visible to clients:

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: private, max-age=30
ETag: "task_123:v7"
X-Request-Id: req_9c6

{"id":"task_123","title":"Write docs","status":"open"}
```

3.49 Cookies and sessions

If you use cookies, set `HttpOnly` and `Secure` flags. Consider `SameSite=Lax` or `Strict` to reduce CSRF exposure. Even token-based APIs should document how cookies are handled.

3.50 Redirects

Avoid redirects for API endpoints. They confuse clients and complicate retries. If you must redirect, use 307 or 308 to preserve method semantics.

3.51 Method safety matrix

Document which methods are safe and idempotent. This guides client retries and caching:

Method	Safe	Idempotent
GET	yes	yes
POST	no	no
PUT	no	yes
PATCH	no	depends on semantics
DELETE	no	yes (if defined that way)

3.52 Content negotiation example

If you support multiple formats, show a concrete example:

```
curl -i https://api.example.com/tasks/task_123 \
-H "Accept: application/json"
```

If a client requests an unsupported format, return 406 Not Acceptable.

3.53 Link headers for pagination

Link headers can make pagination easier for clients:

```
Link: <https://api.example.com/tasks?cursor=task_123>; rel="next"
```

If you use Link headers, keep them consistent with the response body.

3.54 Range requests

For large downloads, support **Range** when it makes sense. This allows clients to resume transfers without re-downloading the entire file.

3.55 HTTP error response example

When errors happen, include headers and a structured body:

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/json
X-Request-Id: req_42

{"error":{"code":"validation_error","message":"title is required","field":"title"}}
```

3.56 Throttling strategy

Rate limits should be predictable. Document the limits per endpoint or per client type. If you use dynamic limits, return them via headers so clients can adapt.

3.57 Batch request example

Batch endpoints should document partial failures clearly. An example batch response:

```
{
  "results": [
    {"id": "task_1", "status": "created"},
    {"id": null, "status": "error", "error": "validation_error"}
  ]
}
```

If you allow batch requests, define how you handle retries for partial success.

Checklist: HTTP

- Use methods for intent.
- Pick status codes and stick to them.
- Add pagination before you need it.

Chapter 4

API design

APIs are contracts. Small design mistakes compound when clients and services depend on them.

4.1 Resources and nouns

Model resources as nouns and actions as methods. Avoid verb-heavy endpoints like `/createTask`.

4.2 Stable identifiers

Use opaque, stable identifiers that do not leak internal state. Do not overload identifiers with business meaning.

4.3 Naming conventions

Use consistent naming across resources and fields. Prefer snake case in JSON fields if your ecosystem expects it, or camel case if that is standard. Do not mix conventions within the same API.

4.4 Nested resources

Nested routes express ownership, but can become deep and repetitive. Prefer one level of nesting and expose flat endpoints for cross-resource queries.

4.5 Versioning

Prefer backward-compatible changes and additive fields. Use versioning when you must break clients.

4.6 Versioning strategies

Strategy	Tradeoff
URI versioning	Easy to see, but splits clients across versions
Header versioning	Cleaner URLs, harder to debug
Query versioning	Simple but often abused

4.7 Deprecation

Deprecation is a process, not a flag. Announce, measure usage, provide a migration path, and remove after a clear deadline.

4.8 Error format

Define one error shape across the API.

```
{
  "error": {
    "code": "validation_error",
    "message": "title is required",
    "field": "title"
  }
}
```

4.9 Field naming

Pick a naming convention and apply it everywhere. If you use `created_at` in one response, do not return `createdAt` in another.

4.10 Filtering

Filtering should be predictable and composable. Use query parameters like `status=open` and `owner_id=user_7`.

4.11 Expandable fields

If a resource is large, allow clients to expand specific fields. This avoids repeated follow-up requests while keeping defaults small.

4.12 Partial responses

For large collections, support selecting specific fields. This reduces bandwidth and keeps responses stable under load.

4.13 Idempotent creates

If clients retry creates, accept an idempotency key. Return the same resource if the create already succeeded.

4.14 Pagination and filtering

Use query parameters for filtering and sorting. Keep pagination consistent across endpoints.

4.15 Pagination example

```
{
  "items": [
    {"id": "task_123", "title": "Write docs"}
  ],
  "next_cursor": "task_123"
}
```

4.16 Filtering example

```
{
  "filters": {
    "status": "open",
    "owner_id": "user_7"
  }
}
```

4.17 Sorting

Expose explicit sort fields and directions. If sorting changes the meaning of pagination, use stable cursor-based pagination.

4.18 Partial updates

Define whether PATCH is merge semantics or JSON Patch. Keep it consistent across endpoints.

4.19 Long-running operations

If an operation takes longer than a few seconds, return a job resource. Clients should poll the job status instead of holding the request open.

4.20 Job example

```
{
  "job_id": "job_98",
  "status": "running",
  "result_url": "/jobs/job_98"
}
```

4.21 Resource lifecycle

Define lifecycle events for each resource. For tasks: created, updated, completed, archived. Ensure the API exposes timestamps for these transitions.

4.22 Documentation

Great APIs have short and accurate docs. Document the request and response shape, error codes, and examples.

4.23 Compatibility testing

If multiple teams depend on your API, add contract tests. They catch breaking changes before they reach production.

4.24 Endpoint naming rules

Keep endpoint names simple and stable:

- plural nouns for collections
- singular nouns for specific resources
- avoid verbs in paths

4.25 Change policy

Document how breaking changes are handled. Common policies include a 90-day deprecation window and a migration guide.

4.26 Search endpoints

Search is different from filtering. Expose a dedicated search endpoint when you need text search or relevance ranking.

4.27 Bulk operations

Bulk endpoints reduce round trips but increase complexity. If you add bulk operations, make error handling explicit and predictable.

4.28 Response envelopes

If you wrap responses in an envelope, keep it consistent for all endpoints. Do not mix raw and wrapped responses.

4.29 Field validation

Define validation rules once and reuse them across endpoints. If you allow partial updates, validate only the provided fields but keep invariants.

4.30 Pagination pitfalls

Offset pagination is easy but unstable for changing datasets. Cursor pagination is stable and scales better for large collections.

4.31 Search response shape

Search responses should include result counts and cursors when possible. If counts are expensive, expose approximate counts explicitly.

4.32 Actions and commands

Some operations are not clean CRUD. For actions like `/tasks/:id/complete`, prefer POST to an action subresource. Keep the action names consistent and document the side effects.

4.33 Write semantics

Make the write semantics explicit so clients do not guess.

Method	Semantics
PUT	Replace the full resource representation
PATCH	Apply a partial update with stable merge rules
POST	Create a new resource or trigger an action

4.34 Default limits

Pick default limits that protect the service:

- default page size, for example 50 items
- maximum page size, for example 200 items
- maximum payload size for create and update

If defaults are not stated, clients will assume the largest possible responses.

4.35 Authorization scopes

Define scopes or roles at the API level. For example, a user may read tasks but not update them. Document the scope requirements per endpoint and return consistent errors.

4.36 Client SDKs and examples

If the API is consumed by many teams, offer a small SDK or client library. Even a thin wrapper helps standardize pagination, retries, and error handling. Keep the SDK minimal and generated from the API contract when possible.

4.37 Backward-compatible changes

These changes are safe:

- adding a new field
- adding a new endpoint
- adding an optional query parameter

These changes are breaking:

- renaming fields
- changing field types
- changing pagination strategy

4.38 OpenAPI as a contract

An OpenAPI specification is a living contract. Use it to generate documentation, validate requests, and drive tests. Keep the spec in version control next to the code.

4.39 Public versus internal APIs

Public APIs need more stability and stricter compatibility guarantees. Internal APIs can evolve faster, but they still need clear ownership. Document which endpoints are public and which are internal only.

4.40 Changelog and announcements

APIs need a changelog. Record additive changes, deprecations, and removals. Even a short Markdown changelog reduces surprise for clients.

4.41 Deprecation headers

When an endpoint is deprecated, return a warning header and a link to the replacement. This helps clients discover changes without reading every announcement.

4.42 Error code taxonomy

Define a small, stable set of error codes. Avoid per-endpoint custom codes unless truly necessary. Clients should be able to handle errors generically.

4.43 Consistency across resources

The same concept should look the same everywhere. If `status` is an enum for tasks, do not expose it as a free-text field elsewhere. Consistency reduces client complexity and mistakes.

4.44 Response examples

Include at least one example request and response per endpoint. Examples are the fastest way for clients to integrate correctly.

4.45 API review workflow

Before a new API launches, run a short review:

- check naming and resource shape
- validate pagination and limits
- confirm error codes and edge cases
- verify backward compatibility

4.46 Design workshop

A short design workshop saves weeks of refactors. Gather the backend and client teams and walk through:

- the primary user flows
- the minimal set of resources
- error cases and retries
- expected growth and versioning needs

Document the decisions and turn them into a contract.

4.47 Contract testing loop

Contract tests should run in CI and staging. If a contract test fails, block the deploy until the change is reviewed. This forces breaking changes to be explicit.

4.48 API governance

Governance does not mean bureaucracy. It means a small set of rules that keep the API coherent:

- a single owner for each endpoint
- a consistent error schema
- a known deprecation process

4.49 Client migration strategy

When you must break compatibility, plan migration:

- announce the change and timeline
- provide a compatibility layer or adapter
- track adoption by client
- remove the old path on the announced date

4.50 Field evolution playbook

Fields evolve more often than endpoints. Use a playbook to keep changes safe:

- add the new field and document it
- deploy clients that read the new field
- update writers to populate the new field
- deprecate and remove the old field

4.51 API metrics

Track API usage and error rates by endpoint and by client. Metrics reveal which endpoints are critical and which can be changed safely. Expose the top endpoints in a dashboard.

4.52 Client types

Different clients have different constraints. Design with those constraints in mind:

Client	Constraint
Mobile app	limited bandwidth, frequent retries
Browser	CORS and cookie behavior
Server-to-server	high throughput, strict SLAs

4.53 Discovery and navigation

Provide stable entry points for clients. If you expose a root endpoint or API index, keep it backward compatible. This reduces the risk of hard-coded URLs in clients.

4.54 Case study: mobile client constraints

Mobile clients retry more often due to unreliable networks. They also cache aggressively to reduce bandwidth. Design endpoints so retries are safe and responses are cacheable. If the API is not retry-safe, mobile clients will create duplicate data.

4.55 SDK versioning

If you publish a client SDK, version it separately from the API. The SDK should track breaking changes and provide deprecation warnings. This helps clients update without surprises.

4.56 Error handling examples

Provide example error payloads for common failures:

- validation errors with field-level details
- authorization failures with a stable error code
- rate limit responses with retry hints

4.57 Caching in API design

Caching is a design decision. If you expect heavy reads, include ETag and cache headers in the contract. Document cache lifetimes so clients can rely on them.

4.58 Case study: breaking change rollout

Suppose you need to replace `status` with a richer object. The safe approach is a staged rollout:

- add a new `status_details` field
- update clients to read both fields
- backfill and populate the new field on write
- deprecate `status` after all clients migrate

This avoids hard breaks and gives clients a clear path forward.

4.59 Compatibility testing example

Contract tests can be simple:

- verify required fields in responses
- check status codes for error conditions
- ensure pagination fields are present

Warning: breaking changes by accident

Renaming a field or changing its type is a breaking change. If you are unsure, add a new field and deprecate the old one.

4.60 Example resource representation

```
{
  "id": "task_123",
  "project_id": "proj_7",
```

```
"title": "Write docs",
"status": "open",
"created_at": "2026-02-18T10:00:00Z",
"updated_at": "2026-02-18T12:00:00Z"
}
```

4.61 Review checklist

Before releasing a new endpoint, review:

- status codes and error shapes
- pagination and limits
- authentication and authorization requirements
- backward compatibility with existing clients

Pattern: add fields, do not change meaning

If you need new behavior, add a field instead of changing an existing one. This keeps old clients working.

Checklist: API design

- Use nouns for resources.
- Keep identifiers stable.
- Standardize error responses.

Chapter 5

Data modeling

Data is the backend's most important interface. Schema decisions are long-lived and difficult to change.

5.1 Entities and relationships

Start with the core entities and how they relate. Use a small number of tables with clear ownership.

5.2 Constraints

Use database constraints for uniqueness and referential integrity. Do not rely solely on application checks.

5.3 Constraint examples

Constraints encode invariants directly in the database.

```
ALTER TABLE tasks
  ADD CONSTRAINT tasks_status_check
  CHECK (status IN ('open', 'in_progress', 'done'));
```

```
CREATE TABLE tasks (
  id TEXT PRIMARY KEY,
  project_id TEXT NOT NULL REFERENCES projects(id),
  title TEXT NOT NULL,
  status TEXT NOT NULL,
  created_at TIMESTAMP NOT NULL
);
```

5.4 Indexes

Indexes are a performance contract. Add indexes for your query patterns, not for every column.

5.5 Normalization and denormalization

Normalize to reduce duplication and ensure correctness. Denormalize when read performance is more important than write complexity.

5.6 Data ownership

Each table should have a clear owner in the domain model. Ownership determines which service or module is allowed to write the data.

5.7 Query patterns

Design the schema around the queries you will actually run. If you regularly filter tasks by project and status, index both fields.

5.8 Example query patterns

Query	Suggested index
List tasks by project and status	(<code>project_id</code> , <code>status</code> , <code>created_at</code>)
Fetch task by id	(<code>id</code>)
List recent activity by task	(<code>task_id</code> , <code>created_at</code>)

5.9 Migrations

Schema changes should be backward compatible. Add columns before you use them, and remove them later.

5.10 Migration example

```
BEGIN;  
ALTER TABLE tasks ADD COLUMN priority INTEGER DEFAULT 0;  
CREATE INDEX idx_tasks_project_status ON tasks(project_id, status);  
COMMIT;
```

5.11 Soft deletes

Soft deletes keep history but complicate queries. If you need history, consider an audit table instead of a `deleted` flag everywhere.

5.12 Audit trails

Write audit logs for critical actions like permissions changes and deletions. These logs are essential for debugging and compliance.

5.13 Data retention

Not all data should live forever. Define retention policies for logs, activity, and temporary data.

5.14 Backfills

If you add a new column, you may need to backfill old data. Plan backfills as separate jobs so they do not block user-facing requests.

5.15 Transactions and isolation

Use transactions to keep multi-step changes consistent. Keep them short and avoid long-running queries inside a transaction.

5.16 Read replicas

Read replicas improve scale but introduce replication lag. If you require read-after-write behavior, read from the primary.

5.17 Explain plans

Use query explain plans to understand performance. Add indexes only when they improve the actual query plan.

5.18 Integrity checks

Schedule periodic integrity checks to catch data drift. These are especially important after large migrations or backfills.

5.19 Write amplification

Denormalization and heavy indexing increase write cost. Measure write latency before adding more indexes.

5.20 Schema evolution

Use additive changes first, then migrate clients, then remove old fields. This minimizes downtime and unexpected client failures.

5.21 Partitioning

Partitioning can keep large tables manageable. Choose a partition key that matches query patterns, such as project id or date.

5.22 Backups and recovery

Backups are part of the data model. If you cannot restore data reliably, your model is not production-ready.

5.23 Data locality

Place data close to the services that use it most. Cross-region reads add latency and can create consistency surprises.

5.24 Case study: activity log growth

Activity logs grow faster than core entities. If you store them in the same table as tasks, reads and writes will slow down. A dedicated activity table with time-based partitioning keeps growth manageable.

5.25 Data validation at rest

Use constraints and types to prevent invalid data from being stored. If your database supports it, use check constraints for enums and ranges.

5.26 Null handling

Nulls are easy to add and hard to reason about. Define when null is allowed and what it means. If a field is required for business logic, make it `NOT NULL`.

5.27 Data lifecycle and classification

Not all data is equally sensitive. Classify fields early:

- public metadata (project name)
- internal operational data (status transitions)
- sensitive data (user identifiers, billing details)

Classification drives encryption, access controls, and retention.

5.28 Hot and cold data

Separate hot data from cold data to keep critical queries fast. For example, keep active tasks in a primary table and archive completed tasks. This keeps indexes small and improves cache hit rates.

5.29 Reference data

Some data changes rarely, such as status codes or categories. Model it as reference tables and enforce foreign keys. This reduces duplication and keeps validation consistent.

5.30 Sharding criteria

If the system grows, sharding may be necessary. Pick a key that keeps related data together, such as project id. Avoid sharding on fields that cause cross-shard queries for core workflows.

5.31 Event tables

Event tables store a log of changes instead of only the current state. They are useful for audits and analytics. If you use event tables, plan retention and compaction to control size.

5.32 Change data capture

Some systems need real-time streams of data changes. Use change data capture or an outbox to publish changes to other services. This keeps integration logic out of the primary request path.

5.33 Precision and types

Choose data types that match the domain:

- use `DECIMAL` for money
- use `TIMESTAMP WITH TIME ZONE` for time

- avoid free-text for enums

5.34 Testing with realistic data

Synthetic data that is too small hides problems. Keep a staging dataset that reflects production volume and skew. This helps validate query plans and migration timing.

5.35 Hotspot avoidance

Some keys receive a disproportionate amount of traffic. If a single project or user dominates traffic, you can create hotspots. Mitigate hotspots with sharding, caching, or dedicated queues.

5.36 Index maintenance

Indexes speed reads but slow writes. Periodically review unused or low-value indexes. Drop indexes that no longer align with real query patterns.

5.37 Data fixes and runbooks

Production data will need fixes. Write a runbook for safe data changes:

- define the scope and affected records
- run changes in small batches
- verify metrics and constraints after the change

5.38 PII and compliance

Personally identifiable information requires special handling. Apply stricter access controls and limit exposure in logs. If you must delete user data, plan for hard deletes or anonymization.

5.39 Analytics schema

Operational schemas are not always ideal for analytics. Use a separate analytics schema or warehouse for reporting. This protects the primary database from heavy reporting queries.

5.40 Time-series data

Time-series data like metrics and events grows quickly. Use time-based partitioning and retention policies. Avoid mixing long-term time-series data with transactional tables.

5.41 Foreign keys and deletes

Define delete behavior explicitly. Use `ON DELETE RESTRICT` for critical data and `ON DELETE CASCADE` only when safe. Unexpected cascades can delete far more data than intended.

5.42 Version columns

Version columns make data changes traceable. They support optimistic locking and conflict detection. If you update a record, increment its version and record the timestamp.

5.43 Multi-tenant schemas

For multi-tenant data, choose a strategy early:

- shared tables with a `tenant_id` column
- separate schemas per tenant
- separate databases for large tenants

Each strategy changes operational complexity and isolation.

5.44 Data quality monitoring

Data quality is an operational metric. Track null rates, duplicate keys, and constraint violations. If you do not measure data quality, it will regress silently.

5.45 Archival strategy

Define what happens when data ages out. Archive to cheaper storage or delete after the retention window. Make the archival process visible and reversible.

5.46 Access patterns and tooling

The way you access data shapes the schema. If you use an ORM, understand its query patterns and add indexes accordingly. If you write raw SQL, standardize queries and avoid ad-hoc joins.

5.47 Partitioning example

Partitioning is only useful when it matches access patterns. For a task system, time-based partitioning is common for activity logs:

Table	Partition key
activity_log	month by created_at
tasks	project_id or created_at

5.48 RPO and RTO

Backups should meet business objectives. Define recovery point objective (RPO) and recovery time objective (RTO) per dataset. If you cannot meet them, adjust either the tooling or the expectations.

5.49 Materialized views

If you need expensive aggregates, consider materialized views. They trade storage and refresh complexity for faster reads. Document refresh schedules and acceptable staleness.

5.50 Data governance

Define who can change schema, who can access production data, and how audits are performed. Governance should be lightweight but explicit to avoid accidental exposure.

5.51 Schema documentation

Schema docs should live near the code. At minimum, document table purpose, key fields, and ownership. Good documentation speeds onboarding and reduces misuses.

5.52 Backfill plan example

Backfills should be treated like data migrations:

- add the new column with a default
- deploy code that reads both old and new fields
- run a backfill in batches with progress tracking
- validate counts and constraints
- remove the old field after verification

5.53 Migration phases

Large migrations work best in explicit phases:

Phase	Goal
Prepare	add schema changes and deploy dual-read code
Backfill	migrate existing records in controlled batches
Cutover	switch writers to the new schema
Cleanup	remove deprecated fields and indexes

5.54 Validation queries

After a migration, verify the data with simple queries:

```
SELECT COUNT(*) FROM tasks WHERE priority IS NULL;
SELECT COUNT(*) FROM tasks WHERE project_id IS NULL;
SELECT status, COUNT(*) FROM tasks GROUP BY status;
```

5.55 Sampling for checks

Full scans can be expensive. Use sampling for daily checks and full scans for scheduled audits. Make the sampling strategy part of the runbook.

5.56 Indexing case study

Assume most reads are `GET /tasks?project=id`. An index on `project_id` is required, but adding `status` can improve filtering. Measure the query plan before and after to confirm the index is used.

5.57 Retention and purge jobs

Retention is not just a policy, it is a job. Schedule purge jobs and track their runtime and impact. If a purge job takes hours, it will compete with production traffic.

5.58 Data drift monitoring

Data can drift after migrations and backfills. Monitor for drift with periodic checks:

- compare counts between old and new columns
- validate enums against reference tables
- sample rows for manual verification

5.59 Schema example

Keeping schema examples in the book makes the discussion concrete:

```
CREATE TABLE projects (  
  id TEXT PRIMARY KEY,  
  name TEXT NOT NULL,  
  owner_id TEXT NOT NULL,  
  created_at TIMESTAMP NOT NULL  
);  
  
CREATE TABLE tasks (  
  id TEXT PRIMARY KEY,  
  project_id TEXT NOT NULL REFERENCES projects(id),  
  title TEXT NOT NULL,  
  status TEXT NOT NULL,  
  created_at TIMESTAMP NOT NULL  
);
```

5.60 Index and query example

Indexes should map directly to hot queries:

```
CREATE INDEX idx_tasks_project_status  
ON tasks(project_id, status, created_at);  
  
SELECT id, title  
FROM tasks  
WHERE project_id = 'proj_7' AND status = 'open'  
ORDER BY created_at DESC  
LIMIT 50;
```

5.61 Audit table example

Audit tables preserve history without bloating primary tables:

```
CREATE TABLE task_audit (  
  id TEXT PRIMARY KEY,  
  task_id TEXT NOT NULL REFERENCES tasks(id),  
  action TEXT NOT NULL,  
  actor_id TEXT NOT NULL,  
  created_at TIMESTAMP NOT NULL  
);
```

5.62 Data dictionary example

A short data dictionary clarifies meaning and ownership:

Field	Meaning
projects.owner_id	user who owns the project
tasks.status	lifecycle state of a task
tasks.created_at	time the task was created
task_audit.action	action performed on the task
task_audit.actor_id	user who performed the action

Pattern: model change as an interface change

If a schema change breaks clients, treat it like a breaking API change.

Checklist: data modeling

- Use constraints for integrity.
- Index the queries you actually run.
- Migrate in small, reversible steps.

Chapter 6

Project structure

Structure determines how fast engineers can navigate and change code. A good structure makes boundaries obvious.

6.1 Layers

A practical backend has clear layers:

- API and transport
- business logic
- data access
- integrations

6.2 Example layout

```
api/  
  handlers/  
  routes/  
core/  
  services/  
  domain/  
infra/  
  db/  
  cache/  
  queue/
```

6.3 Dependency direction

Dependencies should point inward. Infrastructure depends on core, not the other way around.

Pattern: keep frameworks at the edge

Frameworks change frequently. Your core logic should not.

6.4 Boundaries and ownership

Structure should reflect ownership. If a team owns a domain, give it a clear module or package. Ownership makes reviews faster and prevents accidental cross-cutting changes.

6.5 Package by feature

Layered structures are common, but feature-based packages can scale better. For example, a `tasks/` module can own its handlers, services, and data access. Pick one approach and apply it consistently.

6.6 Public versus internal code

Not every package should be imported everywhere. Mark internal packages and keep their APIs narrow. This reduces accidental coupling and keeps refactors small.

6.7 Dependency rules

Define explicit dependency rules to keep boundaries stable.

Layer	Allowed dependencies
API	core services, validation, serializers
Core	domain models, interfaces
Infra	databases, caches, external clients

6.8 Configuration layout

Configuration should be centralized and validated. Keep config parsing in one place and pass typed config to other modules. Avoid reading environment variables throughout the codebase.

6.9 Migrations and data access

Keep database migrations close to the data access layer. If schema changes are scattered, they become unreviewable.

6.10 Testing layout

Mirror your code structure in tests. This makes coverage gaps visible and keeps tests discoverable. Separate fast unit tests from slower integration tests.

6.11 Example test layout

```
tests/  
  unit/  
    tasks/  
    projects/  
  integration/  
    api/  
    db/
```

6.12 API boundaries

Expose a small public API from each module. If a module needs internal helpers, keep them private. This limits the blast radius of changes.

6.13 Shared libraries

Shared libraries are useful but dangerous. Only extract a library when multiple teams truly depend on it. Otherwise, prefer duplication over premature abstraction.

6.14 Monorepo and multi-repo

Monorepos simplify refactors and dependency updates. Multi-repos can improve isolation but slow coordinated changes. Pick the model that matches your team size and release workflow.

6.15 Build and tooling

Tooling should be fast and deterministic. Include a single build entry point and a single test entry point. If engineers need to guess the right command, the structure is failing.

6.16 Documentation placement

Place small docs next to the code they describe. An architecture overview belongs at the root, but module docs belong inside modules. This keeps docs current as code changes.

6.17 Interfaces and ports

Define interfaces at the core boundary. Infrastructure implements those interfaces, not the other way around. This keeps the core testable and independent of frameworks.

6.18 Wiring and composition

Centralize wiring in one place. If dependency injection is scattered, tracing the call graph becomes painful. Create a single composition root that instantiates services and adapters.

6.19 Configuration by environment

Keep environment-specific configuration separate from defaults. Validate configuration at startup and fail fast with clear errors. Avoid ad-hoc `if env == prod` checks in business code.

6.20 Module boundaries

Each module should have clear entry points. If a module exposes ten entry points, it is too porous. Define a small public API and keep the rest private.

6.21 Domain model placement

Keep domain models in the core layer. If the domain model lives in the API or data layer, it will be distorted by frameworks. The domain model should be reusable across transports and integrations.

6.22 Example service interface

```
class TaskService:
    def create_task(self, project_id, title, user_id):
        ...
```

6.23 Example adapter

```
class SqlTaskRepository:
    def create(self, task):
        ...
```

6.24 Runtime boundaries

If you have background workers, treat them as first-class modules. Do not hide workers in the API service. Separate deployments reduce risk and make scaling easier.

6.25 Configuration for workers

Workers often need different settings. Keep worker-specific configuration in separate files or sections. This avoids accidental use of API-specific settings in background jobs.

6.26 Code ownership and reviews

Use ownership files to route reviews automatically. Ownership clarifies who can approve changes in critical modules. This is especially valuable in monorepos.

6.27 Directory naming

Use names that describe purpose, not technology. `billing/` is clearer than `stripe/`. If you change providers, the module name should still make sense.

6.28 Avoiding shared state

Global state and singletons make tests brittle. Prefer explicit dependency passing. If you must use globals, isolate them behind clear interfaces.

6.29 Dependency graph checks

Use automated checks to enforce dependency rules. Static analysis or build-time rules can prevent accidental coupling. Manual policing does not scale.

6.30 Example module layout

```
tasks/
  api/
  domain/
  services/
  repo/
  tests/
```


6.31 Monorepo tooling

If you use a monorepo, invest in tooling:

- fast incremental builds
- targeted test runs
- dependency graph visualization

6.32 Release boundaries

Some modules need slower release cycles. Make those boundaries explicit so you can deploy safely. If everything releases together, risk increases.

6.33 What good structure optimizes for

Structure is not a style preference. It optimizes for change speed, risk isolation, and developer cognition. If you can answer *where does this change belong* in under a minute, you are winning.

6.34 Evolution stages

Early projects can be flat because everything changes together. Mid-stage systems need boundaries as teams grow. At scale, boundaries become contracts with explicit owners, versioning, and release controls. Revisit structure at each growth stage rather than locking in the first layout forever.

6.35 Bounded context mapping

Not all domains belong in the same module. Use bounded contexts to map which concepts share language and data. Cross-context data should flow through explicit interfaces rather than implicit imports.

6.36 Module size thresholds

Set a rough size ceiling to trigger a split. For example, if a module exceeds 10 to 15 public entry points, it may be too broad. The threshold is not strict, but it creates a conversation before the module becomes a monolith.

6.37 API versioning layout

If you support multiple API versions, reflect that in the structure. Keep `v1/` and `v2/` handlers separate while sharing core logic. Avoid duplicating entire stacks; route version differences to thin adapter layers.

6.38 Cross-cutting concerns

Logging, metrics, tracing, and feature flags are cross-cutting. Centralize their setup in a shared infrastructure module. Avoid sprinkling ad-hoc initialization throughout feature modules.

6.39 Feature flags

Keep flags in a dedicated package with typed accessors. Do not read flag values directly from the SDK in every handler. This keeps behavior changes observable and testable.

6.40 Configuration hierarchy

Establish a clear order: defaults, environment overrides, secrets. Document the hierarchy and validate it on startup. If configuration is ambiguous, runtime behavior becomes guesswork.

6.41 Data contracts and schemas

If you exchange JSON or events, store schemas in a stable location. Schema folders provide a source of truth for validation and evolution. Treat schema changes as code changes with reviews and tests.

6.42 Operational tooling

Many systems need CLI or admin tooling. Keep these in `tools/` or `admin/` rather than hiding them inside API packages. Clear placement makes audits and access control easier.

6.43 Deployment topology

Structure should mirror how services deploy. If you deploy a worker separately, keep it in its own module with a clear entry point. Shared code can live in `lib/`, but deployment boundaries must remain explicit.

6.44 Dependency inversion in practice

Interfaces belong in the core, implementations in infrastructure. Avoid the temptation to place interfaces with their implementations. This single decision keeps tests fast and makes swapping providers practical.

6.45 Decision logs

Architecture decisions live in a lightweight `docs/adr/` folder. A short decision log explains why the structure is the way it is. This reduces repeated debates and clarifies tradeoffs for new engineers.

6.46 Onboarding path

New engineers should have a path through the repo. Provide a `README` in each major module with its purpose and entry points. Onboarding time is a direct measure of structural clarity.

6.47 Structural refactors

Refactors are easier with checkpoints. Plan a migration path and move code in small steps. Large structural changes without checkpoints often stall and get reverted.

6.48 Enforcement tooling

Automate structure rules. Use static analysis to prevent forbidden imports. If rules are manual, they will eventually be ignored.

6.49 Example dependency rule config

```
rules:
  core:
    allow:
      - core
      - domain
  api:
    allow:
      - core
      - validation
  infra:
    allow:
      - core
      - infra
```

6.50 Case study: split a growing module

A `billing/` module grows into a catch-all for invoices, payments, and tax. Split into `invoices/`, `payments/`, and `tax/` with explicit interfaces. Add a thin `billing/` façade for cross-module orchestration. The split reduces coupling and lets teams deploy changes independently.

6.51 Ownership matrix

Ownership should be explicit and easy to find. Keep a simple ownership matrix for modules and services. It makes review routing and incident response faster.

6.52 Example ownership table

Module	Owner team
tasks	productivity-platform
billing	revenue-systems
notifications	messaging-foundations

6.53 README templates

Module READMEs should follow a common template. Consistency reduces onboarding time and clarifies boundaries.

6.54 Example module README

```
module: tasks
purpose: task lifecycle and assignment
entry_points:
  - api/handlers.py
  - services/task_service.py
```

```
dependencies:
- core/identity
- infra/queue
owner: productivity-platform
```

6.55 Versioned modules

Large domains sometimes need versioned APIs internally. Keep versioned modules side by side rather than deeply nested. Deprecation is easier when old versions are isolated.

6.56 Shared schema registry

If you publish events, use a shared schema registry folder. Keep schemas close to the publisher and replicate to consumers. Schema reviews are part of code review, not an afterthought.

6.57 Build targets

Structure should reflect build targets. If you build a worker and an API, keep distinct entry points and build files. Avoid a single build target that hides multiple services.

6.58 Testing pyramid alignment

The code layout should align with the testing pyramid. Fast unit tests live with core logic. Integration tests mirror API and data access structure. End-to-end tests sit at the top and target deployed boundaries.

6.59 Migration plan for structure changes

When you restructure, define a migration plan with checkpoints. Move one module at a time and keep API compatibility. Track the plan in a short document so the team stays aligned.

6.60 Anti-patterns

Watch for these structural smells:

- circular dependencies between domains
- modules that expose too many entry points
- infrastructure leaking into core logic

6.61 Pattern: facade modules

Facade modules help you expose a stable surface. They hide internal reorganization while keeping callers stable. Use facades when refactors are frequent.

6.62 Pattern: replaceable adapters

Keep adapters thin and replaceable. If an adapter becomes complex, it is usually mixing business logic. Move logic into the core and keep adapters focused on translation.

6.63 Lifecycle of a new module

Start with a narrow scope and a single entry point. Add explicit dependencies and owners. As the module grows, add tests, docs, and release notes.

Checklist: project structure

- Make layers explicit.
- Keep core logic framework-agnostic.
- Limit cross-layer imports.

Chapter 7

Validation and errors

Validation is the first line of defense. If you accept bad input, every downstream system pays the cost.

7.1 Validation strategy

Validate at the edge and again at the core for critical invariants. Use strict schemas for external requests.

7.2 Error shape

Return one error format to clients and log a richer internal error.

```
{
  "error": {
    "code": "invalid_request",
    "message": "title is required",
    "field": "title"
  }
}
```

7.3 Field errors

Expose field errors for forms and batch operations. Keep error codes stable across versions.

7.4 Mapping errors to status codes

Validation errors should be 400. Permission errors should be 403. Missing resources should be 404.

7.5 Where validation lives

Validation happens at multiple layers:

- edge validation for request shape
- domain validation for business rules
- database constraints for invariants

If any layer is missing, invalid data will leak through.

7.6 Schema validation

Use explicit schemas for external inputs. Schema validation should reject unknown fields when you want strict contracts. If you allow unknown fields, document that behavior clearly.

7.7 Partial update validation

For **PATCH**, validate only the fields provided. Do not allow partial updates to break invariants. If a field depends on another field, validate that dependency explicitly.

7.8 Error taxonomy

Keep a small, stable set of error codes.

Code	Meaning
invalid_request	request payload is malformed or missing fields
validation_error	input fails business rule validation
not_found	resource does not exist or is not visible
conflict	request violates a uniqueness or state constraint
rate_limited	request exceeded a rate limit

7.9 Batch validation

Batch operations should return per-item errors. Do not fail an entire batch when only one item is invalid.

```
{
  "results": [
    {"id": "task_1", "status": "created"},
    {"id": null, "status": "error", "error": "validation_error"}
  ]
}
```

7.10 Localization and messages

Error messages are for humans, error codes are for machines. If you localize messages, keep the codes stable across languages.

7.11 Logging and observability

Log validation failures with a consistent structure. This makes it easy to detect which fields cause the most errors. Do not log sensitive values such as passwords or tokens.

7.12 Retry guidance

Validation errors are not retryable. Conflict errors may be retryable with updated input. Rate limit errors should include **Retry-After**.

7.13 Example validation flow

For POST `/projects/:id/tasks`:

- validate JSON shape and required fields
- confirm project exists and membership is valid
- enforce status and date rules
- return a structured error on failure

7.14 Constraint violations

Database constraints are a final safety net. Map constraint errors to stable error codes. Do not leak internal table or column names to clients.

7.15 Consistency with logging

Client errors should be logged at a lower level. Server errors should be logged at error level with full context. Avoid logging full request bodies for privacy reasons.

7.16 Validation for imports

Bulk imports need different validation. Provide an error report with row numbers and field errors. This helps users fix data without guessing.

7.17 Normalization

Normalize inputs before validation when possible. Trim whitespace, normalize case, and parse dates consistently. This reduces surprising validation failures.

7.18 Reserved values

Some values are reserved or system-defined. Validate them explicitly:

- prevent reserved names
- block forbidden status transitions
- enforce unique identifiers

7.19 Error metadata

Error responses should include a request id. If a support ticket comes in, that id should locate the error in logs.

7.20 Example error response with metadata

```
{
  "error": {
    "code": "validation_error",
    "message": "status is invalid",
    "field": "status",
    "request_id": "req_42"
  }
}
```


7.21 Validation in async jobs

Background jobs should validate inputs too. If a job receives invalid data, fail the job and surface the error. Silent failures create data drift.

7.22 Client-side validation

Client-side validation improves UX but is not a security boundary. Server-side validation is the source of truth.

7.23 Idempotency and validation

If a request is retried, validation should be consistent. Do not allow a retried request to pass validation when the first one failed.

7.24 Error contracts

Treat error shapes as part of the API contract. Changing them can break clients just as much as changing fields.

7.25 Validation as a product surface

Users experience validation failures before they see your core features. Clear, precise error messages reduce support load and increase adoption. Treat validation text as part of your product design.

7.26 Field path conventions

Standardize field paths so clients can map them to UI elements. Use dot-notation or JSON pointer and keep it consistent across endpoints. Do not invent new field path formats per API.

7.27 Partial update rules

PATCH endpoints need explicit rules about missing fields. Define which fields can be cleared, which are immutable, and which require validation only when present. Ambiguity here is a common source of regressions.

7.28 Example partial update policy

```
{
  "update_policy": {
    "title": "optional",
    "status": "optional",
    "owner_id": "immutable",
    "due_date": "nullable"
  }
}
```

7.29 Batch validation details

Batch APIs should return per-item errors with stable indexes. Do not stop at the first failure unless the operation is transactional. Return a summary error and per-item details.

7.30 Error category semantics

Clients need to know if an error is fixable. Separate errors into validation, authorization, conflict, and system categories. Expose a stable `type` value for each category.

7.31 Error category table

Category	Client action
Validation	Fix input and retry
Authorization	Request access or credentials
Conflict	Refresh state and retry if still relevant
System	Retry with backoff, or contact support

7.32 Error code ownership

Assign ownership of error codes to modules. If everyone can mint codes, you end up with duplicates. Ownership also helps with documentation and deprecation.

7.33 Localization strategy

Decide whether servers return localized messages or message keys. Message keys keep the API stable and let clients localize independently. If you localize server-side, document supported locales and fallbacks.

7.34 Normalization and preprocessing

Normalize inputs before validation. Trim whitespace, normalize Unicode, and coerce types where appropriate. Validation without normalization leads to inconsistent results.

7.35 Reserved values revisited

Reserve values like `admin` or `root` explicitly. Do not rely on implicit checks scattered across the codebase. Centralize reserved value checks in one validation module.

7.36 Constraint evolution

Constraints change over time. Introduce new constraints in warning mode before enforcing them. This avoids breaking existing clients without notice.

7.37 Async validation

Some validations require external calls. Separate async validation from synchronous checks. Return a distinct error type so clients can handle the difference.

7.38 Logging and privacy

Validation errors can include sensitive data. Log only sanitized versions of fields. Avoid logging full payloads in production.

7.39 Retries and idempotency

Some errors are safe to retry, others are not. Include a `retryable` flag when relevant. If retries are allowed, ensure the operation is idempotent.

7.40 Example error envelope

```
{
  "error": {
    "type": "validation",
    "code": "task.title.too_long",
    "message": "Title must be 80 characters or fewer",
    "field": "title",
    "retryable": false
  }
}
```

7.41 Auditability

Validation rules are part of system policy. Document them and version them. This helps audits and reduces ambiguity in regulated environments.

7.42 Testing validation

Tests should cover boundary values, not just happy paths. Include property-based tests for input ranges where possible. Regression tests for validation are cheap and valuable.

7.43 Validation architecture

Keep validation rules close to the domain model. Transport layers should handle parsing, but core validation should live with business logic. This prevents duplicate logic across HTTP, CLI, and background jobs.

7.44 Strict versus lenient modes

Some endpoints need strict validation, others tolerate extra fields. Define which mode each endpoint uses. Switching modes silently is a breaking change.

7.45 Backward compatibility

Changing validation rules can be a breaking change. If you tighten rules, consider a deprecation window. Provide guidance to clients before enforcement.

7.46 Error documentation

Document error codes and fields. If clients need to guess, they will implement brittle logic. Error docs are part of your API surface.

7.47 CSV import example

CSV imports need row-level feedback. Return a summary and a per-row error list with line numbers. This is often more useful than a single failure message.

7.48 Example import error report

```
{
  "summary": {"processed": 200, "errors": 12},
  "errors": [
    {"row": 14, "field": "email", "code": "invalid_format"},
    {"row": 87, "field": "status", "code": "reserved_value"}
  ]
}
```

7.49 Constraint mapping

Map database errors to user-facing codes. Do not expose SQL errors or internal constraint names. Keep a mapping table per database.

7.50 Example constraint mapping

Constraint	Public code
users_email_unique	conflict.email_exists
tasks_due_date_check	validation.due_date_range

7.51 Security validation

Validate inputs that affect queries or templates. Reject unexpected characters where appropriate. Sanitize output even when validation passes.

7.52 Field metadata

Include metadata for client UX, such as max length or allowed values. Metadata reduces guesswork in clients. Keep metadata versioned to avoid mismatches.

7.53 Error aggregation

Aggregate multiple errors instead of returning one at a time. This improves client efficiency and user experience. Limit the error count to avoid huge payloads.

7.54 Validation pipeline

A simple pipeline keeps logic predictable:

- normalize input
- run schema validation
- run business validation
- enforce persistence constraints

7.55 Rate limit errors

Rate limits are a validation boundary. Return 429 with a clear retry time. Expose a stable error code for rate limit failures.

7.56 Client UX guidance

If possible, include human-friendly messages. Messages should be short and specific. Avoid blaming language such as *invalid* when you can explain the fix.

7.57 State transition validation

Many fields have allowed state transitions. Validate transitions explicitly rather than relying on ad-hoc checks. State validation prevents invalid workflows from entering the system.

7.58 Warning responses

Sometimes you want to accept input but warn about future behavior. Provide warnings separately from errors. Warnings prepare clients for upcoming constraint changes.

7.59 Example warning response

```
{
  "warnings": [
    {"code": "deprecated_field", "field": "legacy_id"}
  ]
}
```

7.60 Help links

Include a stable help link for common error codes. Help links reduce support burden. Keep links versioned when the documentation changes.

7.61 Limits and caps

Validation includes limits like maximum list size or payload size. Return clear errors when limits are hit. Do not silently truncate user input.

7.62 Contract tests

Add contract tests for validation responses. These tests verify that error shapes and codes remain stable. Contract tests prevent accidental breaking changes.

7.63 Numeric precision

Validate numeric precision explicitly. Money values should use integer cents or decimal types. Floating-point errors create subtle data issues.

7.64 Time zones

Date validation must account for time zones. Store timestamps in UTC and validate input offsets. Reject ambiguous local times during daylight transitions.

7.65 Schema versioning

When schemas evolve, version them explicitly. Support multiple schema versions during migrations. Document the version negotiation strategy for clients.

7.66 Error quotas

Limit the number of errors returned for large payloads. Provide a count of suppressed errors to signal incomplete feedback. This keeps error responses fast and predictable.

7.67 Example error quota

```
{
  "error_count": 500,
  "returned_errors": 100,
  "suppressed_errors": 400
}
```

Checklist: validation

- Validate at the edge.
- Keep error shapes consistent.
- Use stable error codes for clients.

Chapter 8

Authentication and authorization

Authn proves who a user is. Authz decides what they are allowed to do. Mixing them creates security gaps.

8.1 Authentication

Common strategies include sessions, API keys, and JWTs. Pick one and document its lifecycle and rotation policy.

8.2 Authorization

Authorization should be explicit and testable. Use roles or permissions and keep them small.

8.3 Resource ownership

Many systems can model access as ownership or membership. Capture it in the data model and enforce it at the boundary.

8.4 Service-to-service auth

Use short-lived tokens and mutual TLS when possible. Avoid long-lived static keys.

Warning: auth in the UI is not enough

Never rely on frontend checks to enforce permissions. Always enforce authorization on the server.

8.5 Token lifecycle

Tokens should be short-lived and rotated regularly. If you use refresh tokens, store them securely and revoke them on logout. Document token expiration and refresh behavior for clients.

8.6 Session management

Session-based auth requires careful cookie settings. Use `HttpOnly`, `Secure`, and appropriate `SameSite` values. Invalidate sessions on password resets or suspicious activity.

8.7 Claims and scopes

JWT claims should be minimal and stable. Scopes are easier to reason about than complex roles. Keep a mapping table between scopes and allowed actions.

8.8 RBAC and ABAC

Role-based access control (RBAC) is easy to operate. Attribute-based access control (ABAC) is more flexible but harder to test. Choose the simplest model that meets your needs.

8.9 Resource checks

Authorization should happen before expensive work. Check ownership or membership before you query large datasets. This reduces data leakage and improves performance.

8.10 Service identity

Treat service identities like users. Each service should have its own identity and permissions. Do not share credentials across services.

8.11 Audit trails

Record authentication and authorization events. Audit logs help you answer who accessed what and when. Keep audit logs immutable and protected.

8.12 Key rotation

Keys and secrets should rotate regularly. Build rotation into the system so it is routine, not an emergency. If rotation requires downtime, the design is fragile.

8.13 Example policy table

Action	Required scope
Read tasks	<code>tasks:read</code>
Create tasks	<code>tasks:write</code>
Delete tasks	<code>tasks:admin</code>

8.14 Error responses

Return 401 for missing or invalid credentials. Return 403 for valid credentials without permission. Do not leak resource existence in authorization errors.

8.15 Password storage

If you store passwords, never store them in plain text. Use modern hashing algorithms and a strong cost factor. Rotate hashing parameters when industry guidance changes.

8.16 MFA and step-up auth

Some actions need stronger authentication. Use step-up authentication for sensitive operations like deletions and billing changes. This reduces the impact of stolen tokens.

8.17 Delegated access

If users authorize third-party access, use explicit scopes and expiry. OAuth-style flows are common for this scenario. Always allow users to revoke access.

8.18 API keys

API keys should be scoped and revocable. Avoid user-level keys for server-to-server traffic. Log key usage so you can detect abuse.

8.19 Rate limiting

Auth endpoints are high-value targets. Apply stricter rate limits to login and token refresh endpoints. Return clear errors and do not leak which part failed.

8.20 Policy evaluation

Authorization checks should be centralized. If each endpoint rolls its own policy, rules will drift. Consider a policy engine when rules become complex.

8.21 Admin actions

Admin operations need extra logging and review. Require elevated scopes and generate audit events. If admin actions are not tracked, incidents are inevitable.

8.22 Service auth example

```
{
  "service_id": "billing-worker",
  "scopes": ["tasks:read", "tasks:write"],
  "issued_at": "2026-02-18T10:00:00Z"
}
```

8.23 Secret storage

Store secrets in a dedicated secret manager. Do not store secrets in code repositories or environment files. Access should be limited to the services that need them.

8.24 Threat model

List the threats you care about:

- credential stuffing
- token replay
- privilege escalation

- insider access

The threat model guides which controls you implement first.

8.25 Identity proofing

Not all users are equal risk. For high-risk flows, add identity proofing such as verified email or phone. For enterprise accounts, consider domain verification before granting admin access.

8.26 Account lifecycle

Accounts have states: active, locked, disabled, deleted. Define how each state affects authentication and authorization decisions. State transitions should be auditable and reversible where possible.

8.27 Password resets

Reset tokens should be short-lived and single use. Do not reveal whether an email exists in the system. Send reset alerts and require re-authentication after password changes.

8.28 Session binding

Bind sessions to device or IP when risk warrants it. If the context changes significantly, require re-authentication. This mitigates stolen session cookies.

8.29 Token revocation

Short-lived tokens reduce risk, but revocation still matters. Maintain a revocation list for compromised tokens. If you cannot revoke, shorten expiry and rely on rotation.

8.30 Scope design

Keep scopes small and action-oriented. Avoid scopes that combine read and write unless necessary. Document scope usage with examples so clients can request the minimal set.

8.31 Fine-grained checks

Role checks alone are not sufficient for multi-tenant systems. Combine role checks with resource ownership checks. This avoids privilege escalation within the same role.

8.32 Policy composition

Complex policies should be composed, not duplicated. Create reusable policy functions like `can_manage_project`. Composition keeps policy logic consistent across endpoints.

8.33 Authorization at the data layer

Defense in depth is valuable. If possible, enforce row-level security or data filters in the database. This reduces the risk of missing checks in the application layer.

8.34 User impersonation

Support impersonation only with strict controls. Require explicit justification and log every impersonation action. Impersonation sessions should have short expiry.

8.35 Audit log structure

Audit logs should include actor, action, resource, and outcome. Use immutable storage or write-once retention when required. Include correlation ids so incidents can be traced end-to-end.

8.36 Key management

Separate keys by environment and purpose. Use asymmetric keys for tokens when possible. Rotate keys on a schedule and on incident response triggers.

8.37 Secret sprawl

Track where secrets are used. Rotate and prune secrets that are no longer referenced. Secret sprawl is a hidden security risk.

8.38 Service identity onboarding

Provision service identities using automation, not manual steps. Automated provisioning reduces mistakes and missing permissions. Store service identity metadata with owners and expiry dates.

8.39 Security testing

Add tests for authorization boundaries. Use negative tests that assert access is denied. Security tests should run in CI, not just as manual checks.

8.40 Auth error responses

Return errors that are informative without leaking sensitive details. Avoid differences between invalid user and invalid password responses. Keep response times consistent to reduce timing attacks.

8.41 Example policy evaluation flow

```
def authorize(user, action, resource):
    if not user.is_authenticated:
        return "unauthenticated"
    if not has_scope(user, action):
        return "forbidden"
    if not owns_resource(user, resource):
        return "forbidden"
    return "ok"
```

8.42 Incident response

Document how to revoke access quickly. Predefine emergency procedures for compromised keys or accounts. Fast response reduces the blast radius of auth failures.

8.43 Single sign-on

Enterprise users often require SSO. Support SAML or OIDC flows with clear documentation. Test SSO flows in staging with real identity providers.

8.44 Provisioning and deprovisioning

User provisioning should be automated when possible. Deprovisioning must be fast and reliable. If a user leaves, access should be revoked across all services.

8.45 Access reviews

Periodic access reviews catch permission creep. Review roles and scopes for privileged users. Automate review reports to reduce manual work.

8.46 Device trust

Some systems require trusted devices. Bind sessions to devices and require re-approval when devices change. Device trust helps reduce account takeover risk.

8.47 Consent and transparency

When access is delegated, capture consent with timestamps. Let users review and revoke consent in a self-service UI. Consent history helps with compliance and user trust.

8.48 Token format stability

Changing token formats can break clients. If you must change them, version tokens explicitly. Provide a clear migration window for old tokens.

8.49 JWT headers and key ids

Include a key id so verifiers can select the right public key. Rotate keys and publish a key set endpoint. Avoid manual key distribution.

8.50 mTLS for internal traffic

Mutual TLS provides strong service identity. It can reduce the need for API keys in internal networks. Use automation for certificate rotation.

8.51 Authorization caching

Authorization checks can be cached for short windows. Cache only when permissions change infrequently. Invalidate caches on role changes to avoid stale access.

8.52 Auth metrics

Track login success rates, token refresh errors, and authorization denials. These metrics reveal attacks and misconfigurations. Alert on unusual spikes in failed auth attempts.

8.53 Policy drift

As systems evolve, policy logic drifts. Keep policy tests in a dedicated suite. If policy logic changes, update the policy documentation too.

8.54 Example access review checklist

- review admin users
- review service account scopes
- remove unused API keys
- confirm MFA enrollment for privileged roles

8.55 Tenant isolation

Multi-tenant systems require strict tenant isolation. Always include tenant checks in authorization rules. Treat missing tenant checks as security bugs.

8.56 Account logout

Lockouts can stop credential stuffing but can also be abused. Use progressive delays and risk scoring. Allow logout recovery with verified identity.

8.57 MFA recovery

Provide recovery codes for MFA. Recovery codes should be single-use and revocable. Encourage users to rotate recovery codes after use.

8.58 Session limits

Limit concurrent sessions for sensitive accounts. Expose session management in user settings. Let users revoke sessions from a trusted device.

8.59 Policy as code

Represent policies in code with tests and code review. Policy changes should follow the same deployment process as other code. Avoid editing policy rules directly in production.

8.60 Credential stuffing defense

Use IP reputation, rate limits, and breached password detection. Combine signals rather than relying on a single control. This reduces both false positives and false negatives.

8.61 Example auth metrics

```
metrics:
  login_success_rate: 0.98
  token_refresh_failures: 12
  authz_denied_rate: 0.03
```

8.62 Tenant admin boundaries

Tenant admins should be limited to their tenant. Cross-tenant access should require a separate global role. This prevents accidental lateral access.

8.63 SCIM provisioning

SCIM enables automated user lifecycle management. If you support SCIM, test both create and deprovision flows. Ensure SCIM actions trigger the same audit logs as manual changes.

8.64 API access versus user access

Service API access often needs different policies than user access. Separate scopes for machine traffic and human traffic. This reduces the risk of over-privileged service accounts.

8.65 Session timeout policy

Define idle and absolute timeouts for sessions. Idle timeouts reduce exposure for unattended sessions. Absolute timeouts prevent indefinite session lifetimes.

8.66 Key rotation playbook

Publish a rotation playbook with clear steps and owners. Practice rotations to avoid surprises during incidents. Rotation should be a routine operation, not an emergency.

8.67 Break-glass access

Emergency access should be rare and controlled. Use a separate break-glass role with short-lived access. Every break-glass action should create an audit event.

8.68 Just-in-time access

Grant elevated permissions only when needed. Just-in-time access reduces the risk of persistent over-privilege. Automate expiry and notify owners when access is granted.

8.69 Session anomaly detection

Detect unusual login patterns such as impossible travel. Flag anomalies for re-authentication or step-up checks. Anomaly detection reduces account takeover risk.

8.70 Security headers

Set security headers to protect session cookies and tokens. Use **Strict-Transport-Security**, **Content-Security-Policy**, and **X-Content-Type-Options**. Headers are part of the auth surface, not just frontend hygiene.

8.71 Example conditional policy

```
policy:
  action: "export_data"
  require:
    - scope: "data:export"
    - mfa: true
    - business_hours: true
```

Checklist: auth

- Separate authentication and authorization.
- Keep permissions minimal and explicit.
- Rotate credentials regularly.

Chapter 9

Observability

You cannot operate what you cannot see. Observability is the difference between guessing and knowing.

9.1 Logs

Logs are for detailed events and debugging. Use structured logs with stable fields.

9.2 Metrics

Metrics summarize system health. Track rate, errors, and duration for each endpoint.

9.3 Tracing

Distributed traces show where time is spent across services. Use tracing to find the true bottleneck.

9.4 Dashboards

Dashboards should answer specific questions:

- Is the system up?
- Is it fast enough?
- Are errors increasing?

Pattern: instrument the edge

Instrument request entry and exit points. This gives you complete coverage of the user-facing path.

9.5 Structured log example

Structured logs should use stable keys:

```
{
  "level": "info",
  "request_id": "req_42",
  "user_id": "user_7",
  "endpoint": "POST /projects/:id/tasks",
  "status": 201,
```



```
"latency_ms": 84
}
```

9.6 RED and USE

The RED method tracks Rate, Errors, Duration. The USE method tracks Utilization, Saturation, Errors. Use RED for request paths and USE for system resources.

9.7 SLIs and SLOs

Define a small set of service level indicators. Latency p95 and availability are common starting points. SLOs are targets, not guarantees.

9.8 Tracing context

Tracing requires consistent context propagation. Include a request id in every log and response. Ensure background jobs carry the same trace context.

9.9 Sampling

Trace sampling reduces overhead but can hide tail issues. Use adaptive sampling to capture slow requests. Keep error traces at a higher sampling rate.

9.10 Alert design

Alerts should map to user impact. Avoid alerting on internal metrics without context. If an alert does not trigger an action, remove it.

9.11 Dashboard layers

Create dashboards at three levels:

- service overview for on-call
- endpoint detail for debugging
- dependency health for incident response

9.12 Log retention

Logs have cost and compliance implications. Define retention policies and restrict access to sensitive logs.

9.13 Incident timeline

Observability is about time. When incidents happen, you need to see the sequence of events. Ensure logs, metrics, and traces share timestamps and identifiers.

9.14 Error budgets

Error budgets are the currency of reliability. If you burn the budget, pause feature work and focus on stability. This keeps operational work aligned with product priorities.

9.15 Examples of SLIs

Common SLIs include:

- availability of `GET /tasks`
- p95 latency for `POST /tasks`
- background job success rate

9.16 Instrumentation coverage

Coverage should include all critical paths:

- API request entry and exit
- database queries
- external API calls
- queue publish and consume

9.17 Metrics naming

Use consistent metric names and labels. Avoid high-cardinality labels like user id. High cardinality makes metrics expensive and hard to query.

9.18 Example metrics

```
{
  "metric": "http_request_duration_ms",
  "labels": {"route": "/tasks", "method": "GET"},
  "value": 42
}
```

9.19 Trace-to-log correlation

Traces are most useful when linked to logs. Include trace and span ids in log fields. This allows fast navigation from a trace to the exact log lines.

9.20 Golden signals

Use the four golden signals:

- latency
- traffic
- errors
- saturation

9.21 Alert fatigue

Too many alerts reduce response quality. Review alerts monthly and delete those that do not trigger action.

9.22 Runbooks

Every alert should link to a runbook. Runbooks should include diagnosis steps and mitigation actions. If a runbook is missing, the alert is incomplete.

9.23 Log levels

Define when to use `debug`, `info`, `warn`, and `error`. If everything is an error, nothing is. Consistent levels help reduce noise and improve incident triage.

9.24 Log hygiene

Logs should be concise and actionable. Avoid dumping entire request bodies. Redact secrets and personal data before logging.

9.25 Correlation ids

Use a single correlation id per request across services. Return it in responses so support can ask for it. Correlation ids turn vague bug reports into precise searches.

9.26 Metrics types

Use counters for rates, gauges for current values, and histograms for latency. Histograms are essential for p95 and p99 calculations. Avoid averaging latency, which hides tail behavior.

9.27 Histogram buckets

Pick buckets that match user expectations. If user timeouts are at 2s, ensure your histogram has buckets around 500ms, 1s, and 2s. Bad buckets make SLOs hard to interpret.

9.28 Trace span design

Create spans for key operations: handler, database, external calls. Do not create a span for every function. Span design is about clarity, not completeness.

9.29 Context propagation

Propagate trace context through queues and background jobs. If you drop context, traces become useless at service boundaries. Ensure context flows through async code paths.

9.30 Metrics cardinality

Cardinality can break your metrics system. Avoid labels like user id, order id, or request id. If you must label by tenant, limit the number of tenants in metrics.

9.31 Exemplars

Exemplars link metrics to trace samples. They help you jump from a p99 spike to a real trace. Use exemplars on latency and error metrics.

9.32 Synthetics

Synthetic probes catch issues before users do. Run a small set of critical flows on a schedule. Treat synthetic failures as early warnings, not as noise.

9.33 Real user monitoring

RUM captures client latency and failures. Client metrics can reveal DNS and network issues that servers cannot see. If you have a frontend, RUM is a complement to server metrics.

9.34 Queue and worker metrics

Background jobs need their own visibility. Track queue depth, job latency, and retries. Alert on growing backlog rather than individual job failures.

9.35 Dependency monitoring

External dependencies should have their own health metrics. Track error rates and latency for each dependency separately. This helps you avoid blaming the wrong subsystem.

9.36 Alert severity

Define severity levels and expected response times. Severity 1 should map to user impact and page on-call. Severity 3 can be ticketed for later investigation.

9.37 On-call handoff

Shift changes should include a brief operational summary. Outstanding alerts and known risks should be documented. Handoffs reduce repeated work and missed context.

9.38 Incident review

Every major incident should produce a timeline and a corrective action list. Use observability data to validate the timeline. Review what was missing in metrics or logs and add it.

9.39 Example alert rule

```
alert: HighErrorRate
expr: rate(http_requests_total{status=~"5.."}[5m]) > 0.05
for: 10m
labels:
  severity: "sev2"
annotations:
  summary: "High 5xx error rate"
```

9.40 Observability maturity

Start with logs and metrics. Add tracing once you have a stable logging and metrics baseline. Avoid over-investing in tracing before you know what you need to trace.

9.41 Cost management

Observability is not free. Set budgets for log volume, metric cardinality, and trace sampling. Cost controls prevent future rollbacks of visibility.

9.42 Time synchronization

All observability depends on accurate time. Ensure systems use synchronized clocks. Without time sync, cross-service timelines are misleading.

9.43 Service maps

Service maps show dependencies and call paths. They are most useful when tied to real traffic data. Keep maps updated or they quickly become stale.

9.44 Health checks

Expose simple health checks for readiness and liveness. Readiness should verify dependencies, liveness should be minimal. Use health checks in deploy pipelines and orchestration systems.

9.45 SLO documentation

Document SLOs alongside the metrics that power them. SLO docs should include the query and the owner. SLOs without owners are just numbers.

9.46 Example SLO document

```
slo: "task_create_availability"
target: 99.9
window: 30d
metric: "http_requests_total"
filter: "route=/tasks method=POST status=2xx"
owner: "productivity-platform"
```

9.47 Sampling strategy

Sampling should reflect business importance. For critical endpoints, sample at a higher rate. For low-value endpoints, sample aggressively to reduce cost.

9.48 Sampling table

Endpoint class	Sampling rate
Checkout and billing	50%
Core CRUD	10%
Background jobs	5%

9.49 Event logging

Events such as user signups and payments deserve dedicated logs. Event logs are different from debug logs and should be structured. Treat events as data, not just text.

9.50 Data retention tiers

Not all logs need the same retention. Keep high-value security logs longer than debug logs. Tiered retention reduces cost without losing critical evidence.

9.51 Privacy and compliance

Observability data can contain personal information. Define what must be redacted at the source. Compliance requirements should drive retention and access controls.

9.52 Chaos drills

Use drills to test observability, not just reliability. If a failure occurs and you cannot see it, instrumentation is insufficient. Drills reveal blind spots before real incidents do.

9.53 Observability reviews

Review observability alongside feature reviews. Ask which metrics, logs, and traces should be added for new features. This keeps instrumentation aligned with product changes.

9.54 Debugging workflow

Start with dashboards, then traces, then logs. A consistent workflow reduces time to resolution. Document the workflow so new on-call engineers can follow it.

9.55 Log sampling

High-volume services may need log sampling. Sample only low-value logs and keep error logs unsampled. If you sample, include the sampling rate in log metadata.

9.56 Instrumentation backlog

Track missing instrumentation in a backlog. Close the loop after incidents by adding the missing signals. Instrumentation work should be planned, not ad hoc.

9.57 Release gates

Set a release gate that checks key SLOs. If error budgets are exhausted, delay the release. Release gates tie observability to operational decisions.

9.58 Alert tuning loop

Alerts should be tuned after incidents. If an alert fired without action, adjust or remove it. Alert tuning is continuous, not a one-time task.

9.59 Dashboard ownership

Every dashboard should have an owner. Without owners, dashboards become stale quickly. Ownership keeps metrics accurate and relevant.

9.60 Burn rate alerts

Use burn rate alerts for SLOs. Burn rates detect fast and slow budget consumption. This prevents surprises at the end of the SLO window.

9.61 Log search practices

Standardize log search queries for common incidents. Store shared queries in runbooks. This reduces time to diagnose recurring issues.

9.62 Exemplar-driven debugging

If you use exemplars, document how to jump from a metric spike to a trace. Exemplar workflows save time during high-pressure incidents. Teach this workflow in on-call training.

9.63 Metric naming conventions

Use a consistent naming scheme for metrics. Include the unit in the name, such as `_seconds` or `_bytes`. Consistent naming reduces confusion across teams.

9.64 SLI calculation example

```
sli:
  name: "task_create_availability"
  good_events: "http_requests_total{route=/tasks,method=POST,status=2xx}"
  total_events: "http_requests_total{route=/tasks,method=POST}"
```

9.65 Error budget policy

Define what happens when the error budget is depleted. Common actions include slowing releases or focusing on reliability work. Policy clarity prevents debates during incidents.

9.66 Playbook template

```
playbook:
  alert: "HighErrorRate"
  first_checks:
    - "check recent deploys"
    - "inspect dependency latency"
  mitigations:
    - "rollback"
    - "disable feature flag"
```

Checklist: observability

- Log with stable fields.
- Measure latency and error rates.
- Add traces for critical flows.

Chapter 10

Performance

Performance is a user feature. Measure before you optimize, but plan for growth.

10.1 Latency budgets

Define a budget per request and per dependency. Keep the sum of all dependencies below the target latency.

10.2 Profiling

Profile the slowest endpoints first. Most latency comes from a small number of queries or slow calls.

10.3 Caching

Cache what is expensive to compute. Use short-lived caches for dynamic data and long-lived caches for static data.

10.4 Pagination and limits

Large payloads are slow and expensive. Use limits and pagination for any list endpoint.

Pattern: measure p95 and p99

Average latency hides tail problems. Track high percentiles to see real user experience.

10.5 Connection pooling

Database connections are a limited resource. Use a pool sized to your database and avoid unbounded concurrency. If the pool is saturated, requests will queue and latency will spike.

10.6 Query optimization

Most performance issues are in queries. Use EXPLAIN plans and add indexes for actual query patterns. Avoid `SELECT *` on large tables.

10.7 N+1 queries

N+1 queries can multiply latency. Use joins or batch loads when listing resources with related data. Measure query counts per request in tests.

10.8 Caching layers

Caching can happen at multiple layers:

- in-memory cache in the service
- distributed cache for shared reads
- HTTP caching for public responses

10.9 Load testing

Load tests should mimic real traffic patterns. Include bursts, not just steady load. Capture p95 and error rates during the test.

10.10 Capacity planning

Estimate peak requests and data growth. Plan for a buffer so you can handle unexpected spikes. Capacity planning is cheaper than emergency scaling.

10.11 Backpressure

When the system is saturated, shed load early. Use rate limits and queue thresholds to keep the system stable.

10.12 Serialization costs

Large JSON payloads cost CPU. Avoid deeply nested responses and allow field selection. Compress large responses if latency budgets allow it.

10.13 Performance regression tests

Track a small set of performance benchmarks in CI. If latency increases beyond a threshold, block the release.

10.14 Example performance budget

Component	Purpose	Budget (ms)
Database read	fetch task list	40
Cache read	project metadata	5
Serialization	build JSON response	15
Network	response delivery	20

10.15 Async offloading

Move expensive work out of the request path. If work can complete later, return a job id and process asynchronously. This keeps latency predictable for user-facing endpoints.

10.16 Compression tradeoffs

Compression saves bandwidth but costs CPU. Use compression for large responses and measure the impact on tail latency.

10.17 Database tuning

Tune connection limits, statement timeouts, and query caches. A fast query plan is often more valuable than more CPU.

10.18 Hot path optimization

Identify the hot path and keep it lean. Avoid unnecessary allocations, logging, and serialization on the hot path.

10.19 Concurrency limits

Unlimited concurrency will collapse under load. Use bounded queues and worker pools. This protects downstream dependencies from overload.

10.20 Index maintenance

Indexes drift as data grows. Periodically review index usage and drop unused indexes. Unused indexes slow writes and waste memory.

10.21 Read replicas

Read replicas can help with scale but introduce replication lag. Use them for eventually consistent reads only. Do not rely on replicas for read-after-write behavior.

10.22 Performance debugging

When latency spikes, isolate the layer:

- check database latency
- check cache hit rates
- check external dependency latency
- check CPU and memory saturation

10.23 Benchmarking

Benchmarks should be repeatable and versioned. Keep a small suite of benchmarks for critical endpoints. If a benchmark regresses, block the release.

10.24 Case study: slow list endpoint

Suppose `GET /tasks` is slow at high volume. Add a covering index, switch to cursor pagination, and cache project metadata. Measure each change to confirm the improvement.

10.25 Throughput and latency

High throughput can still have poor latency. Track both requests per second and latency percentiles. Performance work should improve user experience, not just raw capacity.

10.26 Tail latency drivers

The slowest requests often come from noisy neighbors or cold caches. Identify which dependencies dominate the p99. Reducing tail latency usually produces the biggest user-visible gains.

10.27 Cache invalidation

Cache invalidation is harder than caching. Prefer short TTLs when correctness is critical. For strong consistency, use write-through caches or cache busting on updates.

10.28 Warmup behavior

Cold starts can dominate latency in autoscaled systems. Warm critical caches and connections on startup. Measure cold-start latency separately from steady-state latency.

10.29 Batching

Batching reduces overhead for external calls and database access. Aggregate work when it does not hurt user-visible latency. Use batching windows that are short enough to avoid queue buildup.

10.30 Client timeouts

Set timeouts for every external call. If a call has no timeout, it will eventually hang the hot path. Align timeouts with your latency budget.

10.31 Circuit breakers

Circuit breakers prevent cascading failures. When dependencies fail, open the circuit and degrade gracefully. This stabilizes latency under partial outages.

10.32 Queue sizing

Queues absorb bursts but add latency. Set queue limits that reflect how long users can wait. If the queue is full, fail fast instead of building unbounded backlog.

10.33 Data model choices

Data model decisions affect performance for years. Normalize for write-heavy workloads, denormalize for read-heavy workloads. Measure the tradeoff before committing to a schema change.

10.34 Serialization formats

JSON is easy but not always fast. Consider binary formats for high-volume internal services. If you change formats, document and test compatibility carefully.

10.35 Memory pressure

Memory pressure increases garbage collection and latency. Monitor heap usage and allocation rates. Avoid large object graphs in hot paths.

10.36 Concurrency versus parallelism

More concurrency is not always better. The goal is to keep resources busy without oversubscription. Use load tests to find the knee of the curve.

10.37 Pagination strategy

Offset pagination is simple but slow for large datasets. Cursor pagination is faster and more consistent at scale. Document cursor formats and ensure they are opaque.

10.38 Example cache strategy

```
cache:
  hot_ttl_seconds: 30
  cold_ttl_seconds: 300
  write_through: true
  max_entries: 50000
```

10.39 Performance budgets per tier

Tier	Notes	Budget (ms)
API layer	parsing, auth, validation	20
Database	primary queries	60
External calls	third-party APIs	80
Serialization	response build	20

10.40 Performance review checklist

Before releasing a new feature:

- validate the latency budget impact
- run a representative load test
- check cache hit rates in staging
- verify timeouts and retries

10.41 Cache stampede

High-traffic cache misses can overload databases. Use request coalescing or locking around cache fills. Stagger expirations to avoid synchronized misses.

10.42 Critical path analysis

List the steps on the critical path for key endpoints. Remove any step that does not change the response. Critical path analysis makes optimizations obvious.

10.43 Async IO

Use async IO for high-latency external calls. Avoid blocking threads on long operations. Async IO is most useful when waiting dominates compute.

10.44 Thread and worker pools

Pools should be sized for the workload and dependencies. If pools are too small, latency spikes. If pools are too large, you waste memory and increase contention.

10.45 Load shedding strategies

When under extreme load, shed low-value traffic first. Return graceful errors for non-critical endpoints. Load shedding protects core user flows.

10.46 Autoscaling policies

Autoscaling can hide performance regressions. Scale should not replace optimization. Set scaling policies that align with latency goals.

10.47 Index selectivity

Indexes only help when they filter well. Low-selectivity indexes add overhead without improving reads. Regularly review query plans and index usage.

10.48 Network latency

Network latency adds up across dependencies. Keep calls within regions when possible. Batch remote calls to reduce round trips.

10.49 TLS overhead

TLS handshakes add latency on short-lived connections. Reuse connections and enable keep-alive. Measure TLS overhead in benchmarks for high-traffic services.

10.50 Compression thresholds

Compression should be applied above a size threshold. Below that threshold, CPU costs outweigh savings. Tune thresholds based on payload sizes and latency budgets.

10.51 Performance incident checklist

When latency regresses:

- check recent deploys and feature flags
- compare p95 and p99 trends
- isolate dependency latency shifts
- verify cache hit rate changes

10.52 Example critical path

```
critical_path:
- auth
- validation
- primary_query
- cache_enrichment
- serialization
```

10.53 Database contention

Lock contention can dominate latency. Watch for long-running transactions and hot rows. Shorten transactions and use optimistic concurrency where possible.

10.54 Hot key mitigation

Some keys receive disproportionate traffic. Use request sharding or cache partitioning for hot keys. Hot key mitigation prevents localized slowdowns.

10.55 Write amplification

Writes can trigger multiple secondary updates. Batch writes when possible and avoid unnecessary index updates. Write amplification often appears during peak traffic.

10.56 Profiling in production

Staging rarely matches production load. Use low-overhead profiling in production for short windows. Ensure profiling is gated and safe to enable.

10.57 Example load test plan

```
test:
  duration_minutes: 30
  ramp_users: 1000
  peak_rps: 800
  scenarios:
    - name: "browse"
    - name: "create_task"
    - name: "bulk_import"
```

10.58 Slow query logging

Enable slow query logs with a sensible threshold. Review slow queries weekly and fix the highest impact first. Slow query logs are an ongoing feedback loop.

10.59 Cache consistency

Decide how much staleness users can tolerate. Use cache invalidation when correctness is critical. Document which endpoints may return stale data.

10.60 Sharding considerations

Sharding can increase throughput but adds operational complexity. Shard keys should align with access patterns. Avoid shard keys that require cross-shard joins.

10.61 IO profiling

Disk and network IO often dominate tail latency. Profile IO during load tests to find bottlenecks. If IO is the bottleneck, CPU optimizations will not help.

10.62 Memory leak detection

Memory leaks cause gradual performance degradation. Monitor memory growth and GC pauses over time. Add automated alerts for abnormal memory trends.

10.63 Latency regression triage

When regressions appear, compare before and after traces. Identify which span added time. Make regressions actionable by linking them to code changes.

10.64 Cold cache benchmarking

Benchmark cold cache behavior separately from warm cache behavior. Cold cache tests reveal worst-case latency. Plan for cold-start scenarios in autoscaled environments.

10.65 Client-side caching

Not all caching belongs on the server. Use client caching for static resources and rarely changing data. Document cache headers so clients behave predictably.

10.66 Lock contention profiling

Lock contention can hide under normal load. Use profiling tools that expose lock wait times. Reduce contention with shorter transactions and better indexing.

10.67 SLA versus SLO

SLOs are internal targets; SLAs are contractual. Avoid promising SLAs you cannot measure accurately. Use SLOs to inform realistic SLAs.

10.68 Connection pool tuning

Pool sizes should match database capacity and workload shape. Monitor pool wait time and saturation. If wait time grows, scale or reduce concurrency.

10.69 Connection pool example


```
db_pool:
  max_connections: 50
  min_idle: 5
  acquire_timeout_ms: 200
```

Checklist: performance

- Define latency budgets.
- Profile before optimizing.
- Cache the expensive paths.

Chapter 11

Background work

Background jobs move slow or unreliable work out of the request path. Use them for emails, reports, and heavy processing.

11.1 Queues and workers

A queue decouples producers from consumers. Workers pull jobs and retry failures.

11.2 Retries

Retries should be bounded and idempotent. Always include a backoff strategy to avoid overload.

11.3 Dead letter queues

A dead letter queue captures jobs that consistently fail. It is the safest place to debug without blocking the main system.

11.4 Job idempotency

Use a job key so the same job can run twice without harm. This is critical during deploys and retries.

11.5 When to use background work

Background jobs are best for work that is slow, unreliable, or can be delayed. Do not move user-critical work to a queue just to avoid fixing latency. If the user expects immediate results, keep a synchronous path.

11.6 Queue types

Queues come in multiple forms:

- FIFO queues for ordered processing
- standard queues for throughput
- delayed queues for scheduled retries

Pick the queue type that matches ordering and latency requirements.

11.7 At-least-once delivery

Most queues deliver at least once. Your job handlers must tolerate duplicates. Idempotency is non-negotiable in this model.

11.8 Visibility timeouts

Visibility timeouts define how long a job is locked by a worker. If the timeout is too short, jobs run twice. If it is too long, failed jobs block progress.

11.9 Retry policy

Retries should be explicit and bounded. Use exponential backoff with jitter. A typical retry policy uses 3 to 5 attempts with increasing delays.

11.10 Retry schedule example

```
retry_policy:
  max_attempts: 5
  backoff_seconds: [5, 30, 120, 300, 900]
  jitter: true
```

11.11 Dead letter handling

Dead letter queues should be monitored. Define a process for triage, not just storage. Automate replays when safe, and require manual review for risky jobs.

11.12 Poison pill jobs

Some jobs fail because of data corruption or invalid state. These should be marked as poison pills and quarantined. Do not retry forever; it wastes capacity.

11.13 Job payload design

Keep payloads small and explicit. Include identifiers, not entire objects. Large payloads increase queue cost and reduce flexibility.

11.14 Priority queues

Not all jobs are equal. Use priorities to keep user-facing work ahead of batch processing. If you mix priorities, document the policy clearly.

11.15 Scheduling and cron

Scheduled work should be expressed as recurring jobs. Use a dedicated scheduler rather than ad-hoc timers. Schedulers should support idempotent retries like any other job.

11.16 Rate limiting jobs

External APIs often impose rate limits. Use token buckets or per-provider queues to avoid throttling. Rate limiting inside the worker prevents accidental abuse.

11.17 Concurrency control

Workers should bound concurrency per job type. A single job class can starve others if not controlled. Use separate queues or worker pools when necessary.

11.18 Ordering requirements

If order matters, use FIFO queues and deduplication keys. Ordering reduces throughput, so only require it when needed. Document which jobs require ordering.

11.19 Idempotency keys

Store idempotency keys with a short TTL. Keys must cover all side effects of the job. If a retry happens, reusing the key should result in a no-op.

11.20 Example job record

```
{
  "job_id": "job_123",
  "type": "send_email",
  "payload": {"user_id": "user_7"},
  "idempotency_key": "email_user_7_welcome",
  "attempt": 2
}
```

11.21 Backpressure

If the queue grows too fast, slow down producers. Backpressure keeps the system stable during spikes. Expose queue depth as a metric to producers.

11.22 Scaling workers

Scale workers based on queue depth and processing time. Fast jobs need more workers than slow jobs. Autoscaling should consider both backlog and throughput.

11.23 Job timeouts

Set execution timeouts for jobs. If a job exceeds its timeout, fail it and retry if safe. Timeouts prevent long-running jobs from blocking workers.

11.24 Cancellation

Some jobs should be cancelable. Support cancellation by checking a cancel flag at safe points. Cancellation is valuable for user-driven actions.

11.25 Bulk processing

Bulk jobs should process in chunks. Chunking allows partial progress and easier retries. Track progress and resume from the last successful chunk.

11.26 Observability for jobs

Record job metrics:

- queue depth
- job latency
- failure rate
- retries per job

These metrics reveal saturation and stability issues.

11.27 Example job metrics

```
metrics:
  queue_depth: 1200
  job_latency_p95_ms: 800
  retry_rate: 0.07
```

11.28 Security considerations

Jobs often handle sensitive data. Do not log full payloads. Use scoped credentials for workers and rotate them regularly.

11.29 Consistency with the request path

Background jobs should not bypass business rules. Reuse core domain logic where possible. If background logic diverges, data corruption is likely.

11.30 Job lifecycle

Jobs should have clear states: queued, running, succeeded, failed. Persist state transitions for debugging and audits. State visibility makes operations predictable.

11.31 Leasing and locks

Workers should acquire a lease when they start a job. Leases prevent multiple workers from running the same job. If a worker dies, the lease expires and the job can be retried.

11.32 Exactly-once illusion

Exactly-once processing is rare in practice. Aim for at-least-once with idempotency. If you need exactly-once outcomes, use deduplication and idempotent writes.

11.33 Deduplication storage

Deduplication keys need storage. Use a small table or cache keyed by idempotency key. Set TTLs based on retry windows and business rules.

11.34 Payload versioning

Job payloads evolve. Include a `payload_version` field and handle old versions. Versioning prevents deploys from breaking queued jobs.

11.35 Schema validation

Validate job payloads before enqueueing. Reject invalid jobs early to avoid poisoned queues. Schema validation should be shared between producers and consumers.

11.36 Job state table

State	Meaning
queued	waiting to be processed
running	currently being handled
failed	exceeded retry policy
succeeded	completed without error

11.37 Fanout and chaining

Fanout jobs create multiple downstream jobs. Chaining jobs ensures a sequence of dependent steps. Both patterns need clear error handling and rollbacks.

11.38 Workflow orchestration

Complex workflows benefit from orchestration. Use a workflow engine if you need stateful retries and branching. Ad-hoc workflows grow fragile quickly.

11.39 Backfills

Backfills are large background jobs that rebuild data. Run backfills in controlled batches to avoid overload. Track progress and support pause and resume.

11.40 Reprocessing policy

Sometimes you need to replay jobs after bugs. Store enough metadata to reprocess safely. Document which job types can be replayed.

11.41 Priority inversion

Priority queues can suffer inversion. If low-priority work blocks high-priority jobs, split queues. Separate worker pools reduce interference.

11.42 Job sharding

Shard heavy jobs by tenant or region. Sharding improves parallelism and reduces contention. Shards should align with data locality when possible.

11.43 Storage considerations

Job queues are not long-term storage. If you need long retention, persist job metadata elsewhere. Queues should be treated as transient buffers.

11.44 Operational tooling

Operators need tools to inspect and requeue jobs. Provide a safe admin interface for job operations. Manual fixes should be logged and auditable.

11.45 Example job admin actions

```
admin_actions:
- replay_job
- cancel_job
- move_to_dlq
```

11.46 Job SLAs

Background work still has latency expectations. Define an SLA for job completion. If jobs exceed the SLA, it should trigger investigation.

11.47 Job latency tracking

Track end-to-end job latency from enqueue to completion. Separate queue wait time from processing time. This helps determine whether the bottleneck is capacity or code.

11.48 Multi-tenant fairness

Large tenants can monopolize queues. Use per-tenant rate limits or weighted queues. Fairness prevents smaller tenants from being starved.

11.49 Routing by job type

Separate queues by job type when workloads differ. CPU-bound and IO-bound jobs should not share the same pool. Routing simplifies tuning and scaling.

11.50 Blue/green for workers

Worker deploys can cause job failures. Use blue/green or rolling deploys for worker fleets. Drain jobs before terminating instances when possible.

11.51 Graceful shutdown

Workers should finish the current job before shutting down. Respect the visibility timeout and release leases cleanly. Abrupt termination causes retries and duplicates.

11.52 Poison pill workflow

Define a workflow for poison pills:

- quarantine the job
- inspect payload and logs
- patch data or logic
- replay safely

11.53 Example job router

```
def route(job):
    if job.type in {"send_email", "send_sms"}:
        return "notifications"
    if job.type in {"generate_report"}:
        return "reports"
    return "default"
```

11.54 Backfill safety

Backfills should be resumable. Store checkpoints after each batch. If a backfill fails, resume rather than restarting.

11.55 Costs and limits

Queues and workers have cost profiles. Track cost per job type. Cost visibility helps prioritize optimizations.

11.56 Side effects

Jobs often trigger side effects like emails or external API calls. Design side effects to be idempotent or safe to repeat. If the side effect is not idempotent, store a side-effect record first.

11.57 Checkpointing

Long-running jobs should checkpoint progress. Checkpointing allows retries to resume rather than restart. Store checkpoints in durable storage.

11.58 Batch windowing

Batch processing benefits from windowing. A small window reduces latency, a large window reduces overhead. Choose a window size based on user expectations.

11.59 Dedupe windows

Deduplication should have a window. A short window reduces storage cost but allows rare duplicates. A long window increases cost but improves safety.

11.60 Example dedupe record

```
{
  "idempotency_key": "email_user_7_welcome",
  "created_at": "2026-02-18T10:00:00Z",
  "expires_at": "2026-02-25T10:00:00Z"
}
```

11.61 Alerts for queues

Alert on queue depth and job age. Queue depth indicates capacity issues. Job age indicates latency and SLA risk.

11.62 Example queue alert

```
alert: QueueDepthHigh
expr: queue_depth > 5000
for: 10m
labels:
  severity: "sev2"
```

11.63 Job SLAs table

Job type	Purpose	SLA (min)
send_email	user notifications	5
generate_report	analytics export	30
reindex_search	rebuild index	120

11.64 Payload size

Large payloads slow queues and increase cost. Compress payloads only when necessary. Prefer storing large objects in a database and referencing them by id.

11.65 Job testing

Background jobs should have unit and integration tests. Test idempotency and retry behavior explicitly. Jobs are part of your production surface and deserve the same rigor.

11.66 Schema registry

If you have many job types, use a schema registry. Schema registry helps validate payloads across teams. It prevents breaking changes to job contracts.

11.67 Reconciliation jobs

Reconciliation jobs fix drift between systems. Run them on a schedule and log corrections. Reconciliation reduces long-term data inconsistency.

11.68 Outbox-driven jobs

Use outbox tables to trigger background work reliably. The outbox ensures jobs are not lost during failures. Outbox-driven jobs reduce dual-write risks.

11.69 Partitioning queues

Partition queues by key for fairness. Partitions allow parallelism without losing ordering. Choose partition keys based on tenant or resource id.

11.70 Example partition config

```
queue:
  partitions: 8
  partition_key: "tenant_id"
```

Checklist: background work

- Keep jobs idempotent.
- Use retries with backoff.
- Monitor queue depth.

Chapter 12

Consistency

Distributed systems trade consistency for availability and latency. You must choose where to be strict and where to be flexible.

12.1 Transactions

Use transactions for multi-step writes that must be atomic. Keep transactions short to avoid lock contention.

12.2 Read-after-write

If the user must see a change immediately, ensure read-after-write consistency. Use primary reads or cache invalidation where needed.

12.3 Eventual consistency

Eventual consistency is fine for analytics and notifications. Document where lag is acceptable.

12.4 Outbox pattern

Write events to the database within the same transaction. Then publish them asynchronously to a queue.

Pattern: consistency by intent

Pick consistency levels by user expectation, not by technical convenience.

12.5 Consistency models

Common models include strong consistency, eventual consistency, and causal consistency. Strong consistency simplifies reasoning but can reduce availability. Eventual consistency scales well but requires careful user communication.

12.6 Isolation levels

Databases provide isolation levels such as read committed and serializable. Higher isolation prevents anomalies but can reduce throughput. Pick the lowest isolation that still preserves

invariants.

12.7 Read your writes

Users expect to see their own changes immediately. Route their reads to primary replicas when needed. If a cache sits in front, invalidate or bypass it for these reads.

12.8 Monotonic reads

Monotonic reads prevent data from appearing to move backward. They are essential for timelines and user-facing counters. Use session-aware routing when possible.

12.9 Write skew

Write skew can occur with concurrent transactions. If it matters, use serializable isolation or explicit locks. Document which invariants require stronger isolation.

12.10 Quorum reads and writes

Distributed stores often use quorums. Define read and write quorum sizes explicitly. Quorums can balance latency and consistency.

12.11 Replication lag

Replication lag makes reads stale. Measure lag and expose it in monitoring. If lag exceeds a threshold, route reads to primaries.

12.12 Cache consistency

Caches are a common source of inconsistency. Use cache invalidation on writes or short TTLs. Document which endpoints may serve stale data.

12.13 Dual writes

Writing to two systems risks divergence. Avoid dual writes when possible. If unavoidable, use an outbox and replayable events.

12.14 Outbox processing

Outbox processing guarantees that events are published. Use a background worker to read the outbox and publish events. Ensure the outbox is idempotent on retries.

12.15 Conflict resolution

Conflicts are inevitable in eventually consistent systems. Define how conflicts resolve: last-write-wins, merge, or rejection. Make the policy explicit and test it.

12.16 Versioning for writes

Use version numbers or etags to detect concurrent updates. Reject writes with stale versions to prevent lost updates. Clients can retry with fresh data.

12.17 Example optimistic concurrency

```
{
  "id": "task_7",
  "version": 12,
  "title": "New title"
}
```

12.18 Sagas

Sagas coordinate multi-step workflows across services. Each step has a compensating action. Sagas trade immediate consistency for eventual convergence.

12.19 Idempotency and consistency

Idempotency reduces the impact of retries. If operations can be replayed, consistency errors are easier to recover. Idempotency keys should be part of the API contract.

12.20 Consistency budget

Define a maximum acceptable lag for eventual consistency. If lag exceeds the budget, treat it as an incident. Consistency budgets align product expectations with engineering limits.

12.21 Consistency examples

Use case	Consistency choice
Payment processing	strong consistency
Activity feed	eventual consistency
Permissions updates	strong or causal consistency

12.22 Testing consistency

Test for consistency under concurrency. Use stress tests to reproduce race conditions. Add integration tests for known anomalies.

12.23 CAP tradeoffs

The CAP theorem forces tradeoffs between consistency and availability. Distributed systems can only fully satisfy two of the three in a partition. Make the tradeoff explicit for each critical workflow.

12.24 Consistency and latency

Stronger consistency often increases latency. If users tolerate a short delay, eventual consistency may be acceptable. Latency budgets should inform consistency choices.

12.25 Session guarantees

Session guarantees include read-your-writes and monotonic reads. They improve user experience without requiring global consistency. Implement them by routing reads to primaries for a session.

12.26 Anomaly table

Anomaly	Description
Dirty read	read uncommitted data
Non-repeatable read	same query returns different results
Phantom read	new rows appear during a transaction

12.27 Read repair

Read repair fixes inconsistent replicas on read. It is useful in eventually consistent systems. Read repair should be throttled to avoid write storms.

12.28 Quorum example

```
replication:
  replicas: 5
  read_quorum: 3
  write_quorum: 3
```

12.29 Multi-region consistency

Multi-region deployments add latency to strong consistency. Use regional primaries for latency-sensitive workflows. Cross-region replication should be treated as eventually consistent.

12.30 Leader election

Leader election defines where writes go. If the leader changes, clients must handle transient errors. Leader election failures are a major source of inconsistency.

12.31 Clock skew

Distributed systems rely on time. Clock skew can cause stale writes or out-of-order events. Use synchronized clocks and avoid relying on timestamps for ordering.

12.32 Idempotent events

Events should be idempotent to handle retries. Idempotent events reduce inconsistency from duplicate processing. Include event ids and dedupe in consumers.

12.33 Conflict example

```
{
  "conflict": {
    "field": "status",
    "expected": "open",
    "actual": "closed"
  }
}
```

12.34 Consistency communication

Users need to know when data is eventually consistent. Expose messaging in the UI where appropriate. Unexplained lag feels like a bug.

12.35 Causal consistency

Causal consistency preserves the order of related operations. It is weaker than strong consistency but stronger than eventual consistency. Causal consistency is often enough for user-facing timelines.

12.36 Search indexes

Search indexes are usually eventually consistent. Document indexing lag and provide refresh options where needed. If search is critical, prioritize index updates.

12.37 Materialized views

Materialized views improve read performance but add consistency lag. Define how often views refresh. Monitor view staleness as a metric.

12.38 Read models

CQRS uses separate read models and write models. Read models are often eventually consistent. Use CQRS only when complexity is justified by scale.

12.39 Pagination consistency

Pagination can surface inconsistent results. Cursor pagination reduces drift across pages. Document whether pagination is stable or eventually consistent.

12.40 Example outbox flow

```
outbox:
  write: "transaction writes event row"
  publish: "worker reads and sends event"
  mark: "event marked delivered"
```

12.41 Consistency in APIs

APIs should document consistency guarantees. If a field is eventually consistent, say so. Consistency documentation reduces support load.

12.42 Write amplification

Strong consistency can increase write amplification. Extra writes affect throughput and latency. Measure the cost before enforcing stricter consistency.

12.43 Linearizability

Linearizability provides a strong guarantee that operations appear instantaneous. It simplifies reasoning but can add latency. Use it for critical invariants such as payments and permissions.

12.44 Causal ordering

Causal ordering ensures related events are seen in order. It is useful for collaboration features and feeds. Causal ordering can be implemented with session tokens.

12.45 Version vectors

Version vectors track causality across replicas. They are more complex than single version numbers. Use them when you need conflict detection across replicas.

12.46 Consistency in caches

Caches can violate consistency even when the database is strong. Invalidate caches on writes or use write-through caches. Monitor cache hit rate and staleness.

12.47 Example cache policy

```
cache_policy:
  write_through: true
  ttl_seconds: 30
  stale_ok_seconds: 5
```

12.48 Two-phase commit

Two-phase commit provides atomicity across systems. It is complex and can block when participants fail. Use it only when the business requires strict atomicity.

12.49 Sagas versus 2PC

Sagas are easier to scale but are eventually consistent. 2PC is strongly consistent but less resilient. Choose based on failure tolerance and business risk.

12.50 Consistency guarantees table

Guarantee	Typical use
Linearizable	payments, permissions
Read your writes	user settings
Eventual	analytics, notifications

12.51 Case study: search lag

A user updates a title but search results lag for 30 seconds. Document the lag and add a manual refresh option. If lag is unacceptable, prioritize index update paths.

12.52 Conflict resolution strategies

Conflict resolution must be explicit. Common strategies include last-write-wins and merge functions. The chosen strategy should be documented and tested.

12.53 Conflict resolution table

Strategy	Tradeoff
Last-write-wins	simple, can lose updates
Merge	preserves intent, more complex
Reject	safe, requires client retry

12.54 Distributed locks

Distributed locks enforce exclusivity. They can improve consistency but add latency and failure modes. Use locks for short critical sections only.

12.55 Freshness metrics

Expose data freshness as a metric. Freshness tells you how stale the system is. Staleness above a threshold should trigger alerts.

12.56 Example freshness metric

```
freshness:
  metric: "index_lag_seconds"
  target: 30
```

12.57 Case study: dual writes

Two systems are updated in parallel and drift over time. Introduce an outbox and replayable events. The outbox becomes the source of truth for synchronization.

12.58 Replication topologies

Single-leader replication favors strong writes. Multi-leader replication improves availability but increases conflicts. Choose the topology that matches your consistency needs.

12.59 Lag-based routing

If replica lag exceeds a threshold, route reads to primaries. Lag-based routing improves read-your-writes behavior. Expose lag metrics to operators.

12.60 Consistency in batch jobs

Batch jobs may read stale data. Document the expected lag for batch outputs. If lag is too high, tighten data refresh policies.

12.61 Consistency testing strategies

Use concurrent test runs to detect race conditions. Test with artificial delays to expose hidden ordering issues. Consistency bugs often only appear under load.

12.62 Example replication config

```
replication:
  mode: "single_leader"
  replicas: 3
  lag_alert_seconds: 5
```

12.63 Write fences

Write fences protect against stale leaders. They ensure only the current leader can commit writes. Fences reduce split-brain inconsistencies.

12.64 Client tokens

Client tokens can encode session consistency requirements. Tokens allow clients to request stronger reads when needed. They are useful for read-your-writes without global consistency.

12.65 Consistency in pagination

Pagination should be consistent across pages. Cursor pagination is more stable than offset pagination. If consistency is weak, communicate it in API docs.

12.66 Read barriers

Read barriers ensure readers observe recent writes. They can be implemented with session tokens or cache bypass. Read barriers improve user experience for critical paths.

12.67 Consistency in analytics

Analytics pipelines are often eventually consistent. Communicate expected delay to stakeholders. If analytics drives user-facing features, tighten the pipeline.

12.68 Staleness budgets

Define a staleness budget per data type. If staleness exceeds the budget, raise an alert. Budgets align expectations with system behavior.

12.69 User-visible consistency

User-facing features should prefer stronger consistency. If a delay is acceptable, communicate it explicitly. Hidden inconsistency feels like a bug.

Checklist: consistency

- Use transactions for critical state.
- Document eventual consistency delays.
- Avoid long-running locks.

Chapter 13

Reliability

Reliability is about the system behaving well under stress. It is a product feature, not just an operational concern.

13.1 Timeouts

Every network call must have a timeout. No timeout means a slow failure becomes an outage.

13.2 Retries

Retries can help or hurt. Only retry on safe operations and use exponential backoff.

13.3 Circuit breakers

Circuit breakers prevent repeated failures from cascading. Open the circuit when a dependency becomes unreliable.

13.4 Bulkheads

Isolate critical resources so one component cannot starve another. Use separate connection pools or worker queues.

13.5 Graceful degradation

When dependencies fail, degrade gracefully. Serve cached data or reduced functionality. Degradation keeps the core user path working.

13.6 Error budgets

Reliability goals should be measurable. Define an error budget and track consumption. If the budget is burned, slow releases and fix reliability gaps.

13.7 Health checks

Use liveness and readiness checks. Readiness should verify critical dependencies. Liveness should be minimal to avoid restart loops.

13.8 Load shedding

When overload hits, shed low-value traffic first. Return fast errors for non-critical endpoints. Load shedding prevents total collapse.

13.9 Rate limiting

Rate limits protect the system from spikes. Use per-user and per-tenant limits when appropriate. Expose limits in responses to support client backoff.

13.10 Fallbacks

Fallbacks are precomputed or cached responses. Use fallbacks for non-critical data. Document when a fallback is acceptable.

13.11 Redundancy

Single points of failure kill reliability. Use redundant instances and multiple availability zones. Redundancy should include databases and caches, not just stateless services.

13.12 Disaster recovery

Disaster recovery requires tested runbooks. Backups are useless without restore drills. Define RTO and RPO targets and measure them.

13.13 Change management

Most incidents come from changes. Use canaries and progressive rollouts. Rollback should be fast and rehearsed.

13.14 Incident response

Incident response needs clear roles. Define incident commander, communications, and operations roles. Practice incident response with game days.

13.15 Post-incident reviews

Post-incident reviews focus on learning, not blame. Track corrective actions and verify completion. Reliability improves only when fixes are implemented.

13.16 Reliability metrics

Track service availability, error rates, and latency. Use SLOs to measure user impact. Metrics should be tied to alerts with clear actions.

13.17 Example reliability targets

Service	SLO	Target
Task API	availability	99.9%
Billing API	availability	99.95%
Worker queue	job success rate	99.5%

13.18 Chaos testing

Chaos tests reveal hidden dependencies. Run controlled failure scenarios in staging. Use chaos tests to validate runbooks and alerts.

13.19 Capacity management

Reliability depends on headroom. Keep enough capacity to handle spikes. Capacity reviews should happen regularly.

13.20 Example circuit breaker config

```
circuit_breaker:
  failure_threshold: 0.2
  window_seconds: 60
  open_seconds: 30
```

13.21 Timeout budgets

Timeouts should align with latency budgets. If a dependency times out after your SLO, it is too slow. Timeouts should be short and explicit.

13.22 Retry budgets

Retries increase load and latency. Define a retry budget so a spike does not overload dependencies. Retry budgets are a key part of resilience engineering.

13.23 Dependency isolation

Critical dependencies should be isolated with bulkheads. Non-critical dependencies should fail open when possible. Isolation prevents cascading failures.

13.24 Multi-region failover

Failover should be automatic and tested. DNS or traffic management must support fast switching. Regular failover drills reduce surprises.

13.25 Data durability

Durability is part of reliability. Use backups with verified restore procedures. Measure backup success and restore time.

13.26 Service ownership

Reliability requires clear ownership. Every service should have an on-call owner. Ownership clarifies who responds during incidents.

13.27 Monitoring maturity

Start with core signals: latency, errors, saturation. Add traces once logs and metrics are stable. Instrumentation gaps should be tracked like bugs.

13.28 Runbook example

```
runbook:
  alert: "HighErrorRate"
  steps:
    - "check recent deploys"
    - "inspect dependency latency"
    - "rollback if needed"
```

13.29 Failure mode catalog

Maintain a catalog of known failure modes. Document mitigations and detection signals. This reduces time to diagnose repeat incidents.

13.30 Reliability reviews

Review reliability after major changes. Confirm SLO impact and capacity headroom. Reliability reviews prevent regressions.

13.31 Dependency SLOs

Dependencies need their own SLOs. Track dependency error rates and latency. If a dependency breaks its SLO, adjust retry and fallback behavior.

13.32 Graceful degradation levels

Define degradation levels to simplify incident response. Each level should describe what features are disabled. This prevents ad-hoc decisions under pressure.

13.33 Degradation table

Level	Behavior
Level 0	full functionality
Level 1	disable non-critical features
Level 2	serve cached data only

13.34 Saturation signals

Saturation includes CPU, memory, and queue depth. Saturation signals are the earliest warning signs. Alert on sustained saturation, not short spikes.

13.35 Progressive delivery

Rollouts should be progressive and reversible. Use feature flags and staged rollouts. Progressive delivery reduces incident blast radius.

13.36 Synthetic monitoring

Synthetic probes detect failures before users do. Run key user flows on a schedule. Treat synthetic alerts as early warnings.

13.37 User-facing status

Status pages build trust during incidents. Update status pages with honest timelines. User communication is part of reliability.

13.38 Example rollout policy

```
rollout:
  canary_percent: 5
  soak_minutes: 30
  expand_percent: 25
```

13.39 Hedged requests

Hedged requests send a second request when the first is slow. They reduce tail latency at the cost of extra load. Use them only for idempotent requests.

13.40 Queue resilience

Queues can buffer spikes but also hide issues. Monitor queue age and failure rate. If queues grow without bound, you are falling behind.

13.41 Feature flags

Feature flags enable rapid rollback. Keep flags short-lived and documented. Stale flags increase complexity and risk.

13.42 Operational readiness

Before launch, confirm runbooks and alerts exist. Run a game day for major features. Operational readiness reduces incidents after launch.

13.43 Resilience scorecard

Use a scorecard to track resilience practices. Scorecards highlight gaps and drive improvements.

13.44 Example resilience scorecard

Practice	Status
Timeouts on all deps	implemented
Retries with jitter	implemented
Runbooks per alert	partial

13.45 Alert quality

Alert quality is more important than quantity. If an alert does not drive action, remove it. Review alerts after each incident.

13.46 Client timeouts

Client timeouts must be shorter than server timeouts. If clients wait longer, they retry and amplify load. Set timeouts explicitly in SDKs.

13.47 Dependency mapping

Map critical dependencies for each service. Dependency maps inform incident response. Keep maps updated as systems evolve.

13.48 Error budget policy

Define actions when error budgets are spent. Examples include feature freezes and reliability sprints. Policies reduce debate during incidents.

13.49 Example error budget policy

```
policy:
  budget_burned: "pause feature releases"
  owner: "reliability"
```

13.50 Incident communications

Clear communication reduces confusion. Provide incident updates on a schedule. Use a template so updates stay consistent.

13.51 Example incident update

```
update:
  time: "10:30 UTC"
  impact: "increased latency"
  mitigation: "rolled back deploy"
```

13.52 Failure modes

Document common failure modes and mitigations. If a failure repeats, add it to the catalog. This shortens future recovery times.

13.53 Failure mode table

Failure	Mitigation
DB saturation	shed load, scale replicas
Cache outage	bypass cache, reduce load
Queue backlog	scale workers, slow producers

13.54 Capacity headroom

Maintain headroom to handle spikes. Headroom targets should be documented and monitored. Running at 100% capacity is a reliability risk.

13.55 Autoscaling policies

Autoscaling should be based on meaningful signals. CPU alone is insufficient for queue-heavy systems. Use queue depth and latency metrics for autoscaling.

13.56 Traffic shaping

Traffic shaping can smooth bursts. Use token buckets for high-traffic endpoints. Shaping reduces sudden load on dependencies.

13.57 Postmortem template

```
postmortem:
  summary: "what happened"
  impact: "user impact"
  root_cause: "primary cause"
  actions:
    - "short-term fix"
    - "long-term fix"
```

13.58 SLO dashboards

SLO dashboards should show current error budget burn. Dashboards make reliability status visible to everyone. Include links to runbooks and recent incidents.

13.59 Dependency budgets

Each dependency should have a latency and error budget. Budgets help teams decide when to fail open or fail closed. If a dependency exceeds its budget, degrade gracefully.

13.60 Queue drain procedures

Queues should be drained during maintenance. Drain procedures prevent job loss during deployments. Document drain steps in runbooks.

13.61 Resilient client behavior

Clients should retry safely with backoff. Expose retry guidance in error responses. Good client behavior reduces load during incidents.

13.62 Latency budgets for dependencies

Each dependency should have a latency budget. Budgets enforce timeouts and fallback behavior. Without budgets, slow dependencies silently erode reliability.

13.63 Reliability guardrails

Guardrails prevent risky changes from shipping. Examples include SLO checks and mandatory canaries. Guardrails reduce accidental regressions.

13.64 Backpressure

Backpressure keeps producers from overwhelming consumers. Use bounded queues and explicit rejection when capacity is full. If you cannot reject, you will eventually fail elsewhere.

13.65 Cold start risk

Cold starts increase latency and error rates. Pre-warm caches and keep a minimum instance baseline. Measure cold start impact in performance tests.

13.66 Maintenance windows

Plan maintenance windows for disruptive work. Communicate clearly and use them sparingly. Maintenance windows reduce surprise outages.

13.67 Reliability debt

Reliability debt accumulates when shortcuts are taken. Track debt like bugs with owners and deadlines. Ignoring debt increases the probability of major incidents.

13.68 User impact mapping

Map critical user journeys to system dependencies. Use this map to prioritize alerts and SLOs. Impact mapping keeps focus on what users feel.

13.69 Stateful service resilience

Stateful systems need extra care during failover. Practice leader election and data rehydration in drills. Failover plans should cover data integrity, not just uptime.

13.70 Escalation policy

Define clear escalation paths for incidents. Escalate based on impact and time without mitigation. Policies reduce confusion during high-severity events.

13.71 Detection latency

Reliability is limited by how fast you detect issues. Measure time to detect and time to mitigate. Short detection latency reduces user impact.

13.72 Failure injection

Failure injection tests real-world behavior. Inject timeouts, errors, and latency into dependencies. Use these tests to validate fallbacks and alerts.

13.73 Reliability and cost

Over-provisioning buys reliability but costs money. Use error budgets to balance cost and risk. The right balance depends on user expectations.

13.74 On-call handoff

On-call handoff should include current incidents and risks. Handoffs reduce repeated work and missed context. Document the handoff in a shared log.

Checklist: reliability

- Time out all dependency calls.
- Retry only idempotent operations.
- Use circuit breakers for flaky dependencies.

Chapter 14

Security

Security is a default path, not a checklist at the end. Small gaps in input handling or permissions become large incidents.

14.1 Input handling

Validate and sanitize input at the boundary. Never pass raw input to SQL or shell commands.

14.2 Secrets

Store secrets in a dedicated manager. Rotate regularly and audit access.

14.3 Dependencies

Track third-party dependencies and patch regularly. Use allowlists for critical integrations.

14.4 Rate limiting

Rate limits protect your system and reduce abuse. Apply limits per user and per IP when needed.

Warning: avoid security by obscurity

Hidden endpoints are not secure endpoints. Enforce authentication and authorization everywhere.

14.5 Threat modeling

Threat modeling identifies the highest risk areas. Start with data flows and trust boundaries. Prioritize fixes based on impact and likelihood.

14.6 Least privilege

Grant the minimal permissions necessary. Use scoped roles for services and users. Review permissions regularly to prevent privilege creep.

14.7 Encryption in transit

Use TLS for all service-to-service and client traffic. Do not allow plaintext connections in production. Rotate certificates regularly.

14.8 Encryption at rest

Encrypt sensitive data at rest. Manage keys in a dedicated key management system. Access to decryption keys should be tightly controlled.

14.9 Security headers

Set standard security headers for web traffic. Use **Strict-Transport-Security**. Use **Content-Security-Policy**. Headers reduce common browser-based attacks.

14.10 Dependency scanning

Automate dependency scans in CI. Prioritize patching critical vulnerabilities. Track exceptions and enforce expiration dates.

14.11 Secrets management

Secrets should never be stored in code. Use a secrets manager with audit logs. Rotate secrets on a schedule and on employee departure.

14.12 Audit logging

Audit logs capture security-relevant events. Store audit logs in immutable storage. Access to audit logs should be limited and monitored.

14.13 Data classification

Classify data as public, internal, confidential, or restricted. Controls should vary based on classification. Document how each class should be stored and accessed.

14.14 Common attack vectors

Plan for common vectors:

- SQL injection
- cross-site scripting
- SSRF
- credential stuffing

Controls should map to these known risks.

14.15 Example security controls

Risk	Control
SQL injection	parameterized queries
Credential stuffing	rate limiting and MFA
SSRF	egress allowlists

14.16 Security testing

Add security tests for critical flows. Use static analysis and dependency scanning. Penetration tests are useful for mature systems.

14.17 Incident response

Define how to respond to security incidents. Include communication plans and regulatory notifications. Practice incident response regularly.

14.18 Key rotation

Rotate keys on a schedule and when incidents occur. Automate rotation to reduce human error. Monitor for failed rotation attempts.

14.19 Secure defaults

Default settings should be secure. Opt-in to risky behavior rather than opt-out. Secure defaults reduce accidental exposure.

14.20 Authentication hardening

Harden authentication with MFA and strong password policies. Detect credential stuffing and apply rate limits. Log and alert on unusual login patterns.

14.21 Authorization boundaries

Authorization checks must enforce tenant boundaries. Use centralized policy evaluation when rules become complex. Authorization errors should not leak resource existence.

14.22 Input validation

Validation prevents injection and unexpected behavior. Use allowlists for critical inputs. Reject input that cannot be parsed safely.

14.23 Secure logging

Logs should exclude secrets and personal data. Use structured logging and redaction. Audit access to sensitive logs.

14.24 Data retention and deletion

Define data retention policies by data class. Implement deletion workflows that truly remove data. Retention policies support compliance and privacy.

14.25 Security reviews

High-risk changes should go through security review. Review threat models and mitigations. Security reviews reduce blind spots in critical flows.

14.26 Example security checklist

- MFA required for privileged roles
- secrets stored in a manager
- dependency scans in CI
- audit logs enabled

14.27 Secure SDLC

Security should be part of the development lifecycle. Add security checks to CI and code review. Security gates prevent risky changes from reaching production.

14.28 Supply chain security

Software supply chains are common attack vectors. Verify dependencies and pin versions. Use signed artifacts when possible.

14.29 Container and host security

Harden containers with minimal base images. Run services as non-root users. Apply host-level hardening and monitoring.

14.30 Network segmentation

Segment networks by trust level. Limit east-west traffic to required services. Segmentation reduces lateral movement during incidents.

14.31 WAF and edge protections

Web application firewalls can block common attacks. Edge protections should include rate limiting and bot detection. Use them as defense in depth, not as the only control.

14.32 Secrets rotation plan

Define who owns secret rotation and when it happens. Rotation should be automated and logged. Test rotation in staging before production.

14.33 Example secret rotation

```
rotation:
  secret: "db_password"
  cadence_days: 90
  owner: "platform"
```

14.34 Data access auditing

Audit access to sensitive data. Access reviews should be periodic. Auditability is key for compliance and incident response.

14.35 CSRF protection

CSRF protection is required for browser-based sessions. Use anti-CSRF tokens and same-site cookies. CSRF controls are part of the auth surface.

14.36 SSRF protection

SSRF attacks can reach internal services. Use egress allowlists and validate URLs. Block access to metadata endpoints.

14.37 Input encoding

Encode output for the correct context. HTML, JSON, and SQL each require different handling. Output encoding is as important as input validation.

14.38 Vulnerability management

Track vulnerabilities and patch timelines. Define SLAs for critical vulnerabilities. Report patch status to stakeholders.

14.39 Incident response runbook

```
incident:
  detect: "alert or report"
  triage: "confirm scope"
  contain: "revoke access"
  eradicate: "patch and rotate keys"
  recover: "restore services"
```

14.40 Security monitoring

Monitor for anomalous access and data exfiltration. Use alerts for spikes in failed logins. Security monitoring should be continuous.

14.41 Data classification table

Class	Example data
Public	marketing pages
Internal	internal docs
Confidential	customer emails
Restricted	payment details

14.42 IAM policy example

```
{
  "action": "read",
  "resource": "billing_invoices",
  "conditions": {"tenant_id": "tenant_7"}
}
```

14.43 Data minimization

Collect only the data you need. Minimization reduces risk and compliance burden. Review data collection regularly.

14.44 Secure deletion

Deletion should remove data and backups when required. Document how deletion propagates across systems. Secure deletion is part of privacy compliance.

14.45 Security training

Engineers should receive periodic security training. Training reduces common mistakes in code and configuration. Security knowledge decays without reinforcement.

14.46 Secure configuration

Misconfiguration is a leading cause of incidents. Use configuration validation and policy checks. Keep defaults locked down in production.

14.47 Access reviews

Access reviews should be periodic and documented. Remove access that is no longer needed. Access reviews reduce long-term exposure.

14.48 Security metrics

Track a small set of security metrics:

- failed login rate
- privileged access changes
- secrets rotation age

Metrics make security posture measurable.

14.49 Example security event

```
{
  "event": "privileged_access_granted",
  "actor": "user_7",
  "scope": "admin",
  "timestamp": "2026-02-18T12:00:00Z"
}
```

14.50 Data residency

Some data must stay within specific regions. Document residency requirements and enforce them. Residency violations can be compliance incidents.

14.51 Secure backups

Backups should be encrypted and access controlled. Test backup restores regularly. Backups are part of your security surface.

14.52 Key management

Keys should be separated by environment and purpose. Use a dedicated key management system. Audit key access and rotation events.

14.53 Security reviews for changes

High-risk changes should require an explicit security review. Track security review approvals in code review tools. Security reviews reduce surprise regressions.

14.54 Configuration drift

Configuration drift can open security gaps. Use configuration as code and policy checks. Drift detection should alert on unauthorized changes.

14.55 Secure defaults checklist

- deny by default for network access
- MFA required for privileged roles
- encryption enabled at rest

14.56 Secrets sprawl

Secrets sprawl happens when credentials are copied across services. Track secrets inventory and owners. Reduce sprawl by centralizing secret access.

14.57 Security posture reviews

Perform periodic security posture reviews. Review logging, access, and patch status. Posture reviews surface slow-moving risks.

14.58 Patch cadence

Define a patch cadence for critical dependencies. Automate updates where possible. Delayed patches are a leading incident cause.

14.59 Session management

Session tokens should be short-lived and rotated on privilege changes. Invalidate sessions on logout and security events. Session design is part of the authentication surface.

14.60 Password storage

Store passwords with a strong adaptive hash and unique salts. Never log or transmit passwords in plaintext. Password storage errors are permanent incidents.

14.61 Token handling

Treat access tokens as secrets with strict scope. Use short expiry and refresh tokens for long-lived sessions. Log token usage anomalies and revoke quickly.

14.62 File upload security

Validate file types and sizes at the edge. Store uploads in isolated buckets with least privilege access. Scan uploads for malware before processing.

14.63 CORS policy

CORS should be explicit and minimal. Avoid wildcard origins for authenticated endpoints. Document allowed origins and review regularly.

14.64 Third-party risk

Vendors can introduce security and compliance risk. Review vendor security posture and data handling. Terminate access when contracts end.

14.65 Secrets in CI

CI systems are a high-risk secrets target. Use short-lived tokens and restrict build permissions. Rotate build credentials regularly.

14.66 Security champions

Security champions create local expertise in teams. They review changes and raise security issues early. Champions improve the speed of security fixes.

14.67 Security SLAs

Define response SLAs for security findings. Critical issues should have fast remediation targets. SLAs ensure security work does not stall.

14.68 Audit readiness

Prepare evidence for audits continuously. Keep policy docs, logs, and approvals organized. Audit readiness reduces disruption during compliance reviews.

Checklist: security

- Validate all input.
- Rotate secrets and audit access.
- Patch dependencies and libraries.

Chapter 15

Testing

Testing is how you keep change safe. Tests should reflect how the system is used, not just how it is built.

15.1 Unit tests

Unit tests validate small pieces of logic quickly. Use them for validation rules and pure functions.

15.2 Integration tests

Integration tests validate the system against real dependencies. Use them for database access and external APIs.

15.3 Contract tests

Contract tests ensure client and server stay compatible. They are valuable in multi-team environments.

15.4 Test data

Use deterministic fixtures. Avoid shared state across tests.

15.5 Test pyramid

Balance unit, integration, and end-to-end tests. Most tests should be fast unit tests. Use end-to-end tests for critical user journeys only.

15.6 End-to-end tests

End-to-end tests verify the full system. They are slow and brittle if overused. Limit them to core flows such as sign-in and checkout.

15.7 Property-based testing

Property-based tests explore many inputs automatically. They are effective for validators and parsers. Use them to uncover edge cases not covered by examples.

15.8 Fuzz testing

Fuzzing sends random input to find crashes. It is useful for parsers and boundary-heavy code. Run fuzz tests on a schedule to find regressions.

15.9 Mutation testing

Mutation testing measures test quality by introducing small code changes. If tests still pass, coverage is weak. Use mutation testing sparingly due to cost.

15.10 Flaky tests

Flaky tests erode trust in the test suite. Quarantine flaky tests and fix them quickly. Track flakiness as a metric.

15.11 Test environments

Keep test environments consistent with production. Use the same database engine and similar configuration. Environment drift causes false confidence.

15.12 CI pipelines

CI should run fast tests on every commit. Longer integration tests can run on a schedule or before release. Fail fast for obvious regressions.

15.13 Test data management

Reset data between tests to avoid cross-test pollution. Use database transactions or containerized databases. Test data should be reproducible.

15.14 Contract testing

Contract tests prevent breaking changes between teams. Store contracts in version control. Fail builds when contracts are violated.

15.15 Mocking strategy

Mock external dependencies that are slow or unreliable. Avoid mocking your own code. Use real dependencies for integration tests.

15.16 Performance testing

Performance tests validate latency budgets. Run them before major releases. Track regressions over time.

15.17 Canary testing

Canary releases are a form of testing in production. Use feature flags and gradual rollouts. Monitor metrics closely during canaries.

15.18 Example test matrix

Test type	Primary goal
Unit	validate logic quickly
Integration	validate dependencies and IO
End-to-end	validate user journeys

15.19 Example test config

```
tests:
  unit: "pytest -m unit"
  integration: "pytest -m integration"
  e2e: "pytest -m e2e"
```

15.20 Snapshot testing

Snapshot tests capture output for regression detection. Use them for stable outputs only. Large snapshots are hard to review and maintain.

15.21 Database migration tests

Migrations should be tested against realistic data sizes. Run migrations on a copy of production schema when possible. Migration failures are costly in production.

15.22 Test isolation

Tests must be isolated to be reliable. Avoid shared global state. Use per-test databases or transaction rollbacks.

15.23 Test coverage

Coverage is a signal, not a goal. Focus on critical code paths and edge cases. High coverage with weak assertions is still risky.

15.24 Data factories

Data factories create realistic test data. Factories reduce duplication and keep tests readable. Keep factories deterministic to avoid flakiness.

15.25 Golden path testing

Ensure at least one golden path test per major feature. Golden paths verify the most common user flows. They are useful as smoke tests in CI.

15.26 Security testing

Security tests validate common attack vectors. Include tests for authorization boundaries. Security tests are critical for high-risk endpoints.

15.27 Test reports

Test reports should be readable and actionable. Include failure context, logs, and artifacts. Poor reports slow down debugging.

15.28 Static analysis

Static analysis catches issues before runtime. Use linters and type checkers where possible. Static analysis complements tests, it does not replace them.

15.29 Test selection

Run only relevant tests when possible. Use change detection to scope test runs. Test selection speeds up feedback without sacrificing coverage.

15.30 Parallelization

Parallel test execution reduces CI time. Ensure tests are isolated before parallelizing. If tests share state, parallelization increases flakiness.

15.31 Release testing

Release testing validates the system in a staging environment. Run a small suite of critical tests before production. Release tests reduce deployment risk.

15.32 Example CI pipeline

```
pipeline:
  stages: [lint, unit, integration, e2e]
  lint: "ruff check ."
  unit: "pytest -m unit"
  integration: "pytest -m integration"
  e2e: "pytest -m e2e"
```

15.33 Test ownership

Assign ownership for critical tests. Owners keep tests updated as the system evolves. Ownership reduces abandoned or stale tests.

15.34 Test strategy document

Maintain a test strategy document per service. It should describe test types and critical flows. The document keeps testing aligned with architecture.

15.35 Test debt

Test debt accumulates when features ship without coverage. Track test debt explicitly and pay it down. High test debt increases release risk.

15.36 Flaky test triage

Create a triage path for flaky tests. Quarantine, investigate, and fix. Do not ignore flakiness, it spreads.

15.37 Non-functional testing

Non-functional tests include performance and reliability tests. Run them for critical services on a schedule. Non-functional testing prevents hidden regressions.

15.38 Contract evolution

When contracts change, update tests in lockstep. Use versioned contracts for breaking changes. Contract evolution should be deliberate.

15.39 Example test strategy outline

```
strategy:
  goals:
    - "fast feedback"
    - "protect critical flows"
  test_types:
    - unit
    - integration
    - e2e
```

15.40 Environment parity

Test environments should mirror production where possible. Differences in databases or caches lead to missed bugs. Document any unavoidable differences.

15.41 Test data seeding

Seed data should be minimal and purposeful. Large seed sets slow tests and hide intent. Use factories for most test data.

15.42 Test containers

Containers can provide consistent dependencies. Use containers for databases and queues in integration tests. Containerized tests reduce environment drift.

15.43 Test stability metrics

Track pass rate and duration trends. Large changes in duration can indicate regressions. Stability metrics help keep the suite healthy.

15.44 Example stability metric

```

stability:
  pass_rate: 0.995
  avg_duration_minutes: 12

```

15.45 Testing in production

Testing in production includes canaries and shadow traffic. Shadow traffic validates behavior without user impact. Always gate production tests with feature flags.

15.46 Contract test example

```

{
  "contract": "tasks.create",
  "request": {"title": "Test"},
  "response": {"id": "task_1", "title": "Test"}
}

```

15.47 Test naming

Clear names make failures easier to diagnose. Include the behavior and context in the test name. Avoid generic names like `test_1`.

15.48 Test refactoring

Tests should be refactored like production code. Remove duplication and improve clarity. Unmaintained tests become liabilities.

15.49 Observability for tests

Collect logs and traces for failing tests. Artifacts speed up debugging. If a test fails without context, fix the test.

15.50 Test environments table

Environment	Purpose
CI	fast feedback
Staging	release validation
Production	canary and shadow traffic

15.51 Data migration testing

Test migrations with realistic data volumes. Validate both forward and rollback migrations. Migration testing prevents production outages.

15.52 Performance regression tests

Keep a small suite of performance tests. Track p95 and p99 latency over time. Block releases on significant regressions.

15.53 Chaos testing in staging

Chaos tests validate system behavior under failure. Run chaos tests in staging before major releases. Chaos testing reveals missing resilience.

15.54 Test suite hygiene

Remove obsolete tests regularly. Obsolete tests waste time and reduce clarity. Hygiene keeps the suite fast and relevant.

15.55 Test data cleanup

Clean up test data after runs. Orphaned data can pollute environments. Cleanup keeps environments stable.

15.56 Test performance budgets

Set budgets for total test runtime. If tests exceed the budget, optimize or split them. Runtime budgets keep CI fast.

15.57 Example test budget

```
test_budget:
  total_minutes: 20
  unit_minutes: 5
  integration_minutes: 10
```

15.58 Test prioritization

Prioritize tests that protect critical revenue paths. Low-value tests can run less frequently. Prioritization keeps CI fast and focused.

15.59 Flake quarantine

If a test flakes, quarantine it and assign an owner. Quarantined tests should not block releases. Track quarantine counts as a health metric.

15.60 Boundary testing

Boundary tests validate behavior at limits. Test max sizes, empty inputs, and invalid types. Boundary tests catch edge cases before production does.

15.61 Concurrency testing

Concurrent access can break otherwise correct logic. Test for races in caches, queues, and database updates. Use deterministic schedulers when possible.

15.62 Clock control

Time-dependent logic needs controlled clocks in tests. Use fake clocks to avoid sleeps and flakiness. Clock control keeps tests fast and reliable.

15.63 Test doubles

Use the right kind of double for the situation. Stubs provide data, mocks verify interactions, fakes provide behavior. Choose doubles that keep tests readable and stable.

15.64 Test doubles table

Double	Purpose
Stub	returns fixed data
Mock	verifies interactions
Fake	simplified behavior

15.65 Schema testing

APIs and events should be validated against schemas. Schema tests catch breaking changes early. Use versioned schemas for evolvable systems.

15.66 Replay testing

Replay tests run production traffic against new versions. They catch behavior drift without user impact. Ensure data is anonymized before replay.

15.67 Data privacy in tests

Never use raw production data in lower environments. Anonymize or synthesize test data. Privacy violations in tests are still violations.

15.68 Test review

Review tests with the same rigor as production code. Reject tests with weak assertions or poor isolation. Test review improves suite quality over time.

15.69 Risk-based testing

Prioritize tests based on business impact. High-risk features need deeper coverage. Risk-based testing keeps efforts aligned with priorities.

15.70 Example risk matrix

Risk level	Testing focus
High	end-to-end plus integration
Medium	integration plus unit
Low	unit only

15.71 API testing

API tests should validate status codes and error shapes. Include authentication and authorization scenarios. APIs are the contract boundary for most systems.

15.72 Queue processing tests

Async workers need tests for retries and idempotency. Validate that duplicates do not corrupt data. Queue tests prevent silent failures in background work.

15.73 Feature flag testing

Test both enabled and disabled flag paths. Ensure flags are removed after rollout completes. Flags are code paths that can rot quickly.

15.74 Rollback testing

Rollbacks should be tested like deployments. Validate that schema changes are backward compatible. Rollback tests reduce risk during incidents.

15.75 Load test fixtures

Use realistic workloads and data distributions. Synthetic uniform loads hide bottlenecks. Load tests should match real usage patterns.

15.76 Test tagging

Tag tests by scope and criticality. Use tags to run a minimal smoke suite quickly. Tags help balance speed and coverage.

15.77 Example test tags

```
tags:
  smoke: ["auth", "checkout"]
  slow: ["e2e", "load"]
```

15.78 Schema migration rehearsal

Run migrations in staging with production-like volumes. Measure duration and verify data integrity. Rehearsals prevent surprises during release windows.

15.79 Eventual consistency tests

Systems with async flows need tests for timing windows. Assert eventual outcomes with bounded retries. Explicitly document expected delays.

Checklist: testing

- Keep unit tests fast.
- Use integration tests for critical flows.
- Keep fixtures deterministic.

Chapter 16

Deployments

Deployments should be repeatable and low risk. If a deploy requires heroics, the system is too fragile.

16.1 Deployment strategy

Choose a strategy that fits your traffic and risk profile. Blue-green and canary deploys reduce blast radius.

16.2 Migrations

Run migrations separately from application rollouts. Use backward-compatible schema changes.

16.3 Rollback

Rollbacks should be safe and fast. Keep previous versions ready to redeploy.

16.4 Configuration

Keep configuration external to the code. Use environment variables or a config service.

Checklist: deployments

- Automate deploys.
- Separate migrations from rollouts.
- Practice rollbacks.

Chapter 17

Architecture choices

There is no perfect architecture. You choose based on team size, domain complexity, and operational maturity.

17.1 Monoliths

Monoliths are simpler to build and deploy. They are often the right choice early on.

17.2 Microservices

Microservices introduce operational and coordination costs. Only split when you need independent scaling or ownership.

17.3 Modular monoliths

A modular monolith keeps one deployable but enforces internal boundaries. It is often the best of both worlds.

17.4 Decision criteria

- team ownership boundaries
- scaling needs
- release cadence
- dependency isolation

Checklist: architecture

- Prefer a monolith until you cannot.
- Split services only with clear ownership.
- Keep boundaries explicit.

Chapter 18

Event-driven systems

Event-driven systems decouple producers and consumers. They can scale well, but they require discipline in schema and delivery guarantees.

18.1 Events and commands

Events describe something that happened. Commands request something to happen. Keep them distinct.

18.2 Schemas

Define event schemas and version them. Use a schema registry if you have multiple consumers.

18.3 Ordering and duplication

Assume events can arrive out of order and more than once. Design handlers to be idempotent.

18.4 Backpressure

Consumers must handle overload gracefully. Use queue limits and retry policies.

Checklist: event-driven

- Version event schemas.
- Design idempotent consumers.
- Expect reordering and duplication.

Chapter 19

Multi-tenancy

Multi-tenant systems serve multiple customers from one codebase. The design choice is how much isolation each tenant receives.

19.1 Isolation models

- shared database, shared schema
- shared database, separate schema
- separate databases per tenant

19.2 Tenant identification

Every request should carry a tenant identifier. Never infer tenant from user data alone.

19.3 Data access rules

Enforce tenant scoping in every query. Prefer database-level row security when possible.

19.4 Noisy neighbors

Large tenants can degrade others. Use quotas and rate limits to protect shared resources.

Checklist: multi-tenancy

- Decide isolation level early.
- Enforce tenant scoping everywhere.
- Protect against noisy neighbors.

Chapter 20

Production readiness

Production readiness is a discipline. A backend is ready when it can be deployed, observed, and operated safely.

20.1 Runbooks

Write runbooks for common incidents. Keep them short and specific.

20.2 Alerts

Alerts should be actionable. Page on symptoms, not on every metric.

20.3 Capacity

Know your limits before you hit them. Track usage and plan for growth.

20.4 On-call

On-call is part of the product. Improve the system when you are paged, not just the alert.

Checklist: production readiness

- Have runbooks for critical incidents.
- Alert on user-visible symptoms.
- Review capacity regularly.

Appendix A

Appendix: checklists

A.1 API design checklist

- Resource names are nouns.
- Errors use a consistent shape.
- Pagination is defined.

A.2 Data modeling checklist

- Constraints enforce integrity.
- Indexes match query patterns.
- Migrations are reversible.

A.3 Operations checklist

- Alerts are actionable.
- Runbooks exist for top incidents.
- Deployments are automated.

A.4 Security checklist

- Input is validated.
- Secrets are rotated.
- Dependencies are patched.

Appendix B

Appendix: templates

B.1 Standard error response

```
{
  "error": {
    "code": "validation_error",
    "message": "title is required",
    "field": "title"
  }
}
```

B.2 Pagination response

```
{
  "items": [
    {"id": "task_123", "title": "Write docs"}
  ],
  "next_cursor": "task_123"
}
```

B.3 SQL migration template

```
BEGIN;
ALTER TABLE tasks ADD COLUMN priority INTEGER DEFAULT 0;
CREATE INDEX idx_tasks_project_id ON tasks(project_id);
COMMIT;
```

B.4 Health check endpoint

```
{
  "status": "ok",
  "version": "1.2.3",
  "uptime_seconds": 12345
}
```

Closing note

Backends are systems. Code quality matters, but the system behaviour matters more. Make tradeoffs explicit, keep interfaces stable, and build feedback loops into the product.